Error Type: APT
Error Related Type: #pragma, Array, Memory, Index, Pointers
Error Related Pragma: '#pragma HLS array_partition'

Error Type Description

Basic Knowledge
The Array Partition Type (APT) error is a specific error that occurs in High Level Synthesis (HLS) code when using the #pragma HLS array_partition directive with an inappropriate partition type. This error is tied directly to the selection of the partitioning type (complete, block, or cyclic) for arrays in the HLS code. You should ONLY take modifications on the type of array partitioning for APT error. If your code does not include this pragma, the APT error does not apply, and you should avoid inserting it unnecessarily.

You should also avoid mixing this type of error with the dimention error, which refers to DID (Dim incorrect dimension) and partition factor error refers to 'FND'(Factor not divisible).

* Causes of APT Error
The APT error is specifically triggered by the choice of partition type when using the #pragma HLS array_partition directive for option `type` or direct statement. This directive allows designers to partition arrays into smaller arrays or individual elements, which is crucial for optimizing memory access and parallelism in hardware. However, selecting an inappropriate partition type can lead to conflicts or inefficiencies in the design, thereby causing an APT error.

* Types of Array Partitioning and Their Relation to APT Error
The #pragma HLS array_partition directive offers three primary types of partitioning for `type=`, each with its unique characteristics and potential pitfalls:

** Complete Partitioning:
This type of partitioning splits the array into its individual elements, effectively creating a separate register for each element. While this maximizes parallel access, it can also lead to excessive resource usage (registers) and complex routing in the hardware. This type of partitioning may less possible to induce memory access-related issue compared to other two.

** Block Partitioning:
Block partitioning divides the array into contiguous blocks of elements, with each block being stored in a separate memory resource (e.g., Block RAM). This method is useful for optimizing memory access patterns where data is accessed in chunks.
An APT error might be encountered if block partitioning is chosen in scenarios where the memory access pattern doesn't align well with the block structure, or if the array size doesn't neatly divide into the specified block sizes (Please also refer to FND (Factor Not Divisible) error). Additionally, misalignment issues may arise if the block partitioning leads to inefficient memory usage or address conflicts.

** Cyclic Partitioning:
Cyclic partitioning interleaves the elements of the array across multiple smaller arrays. For example, in a 4-way cyclic partition, the array's elements are distributed in a round-robin fashion across four smaller arrays. This is particularly useful in loop unrolling scenarios where consecutive accesses are spread across different memory banks.
APT errors can occur if cyclic partitioning is chosen incorrectly, such as in cases where the access pattern doesn't match the cyclic distribution, leading to inefficient memory access and potential conflicts in addressing. For example, using pointer casting to access a cyclic patitioned array in an index-sequential manner, which has address mismatch.

Error Knowledge
* Scenarios Where APT Errors May Occur
Several specific scenarios can lead to APT errors due to the inappropriate choice of partition type:

a. Mismatch Between Partitioning Type and Access Pattern:
If the chosen partition type does not align with the way the array is accessed in the design, it can lead to inefficient memory access, dangling ports, resource overuse, or routing issues, all of which can trigger an APT error that induce bad synthesis design with high latency, even inconsistent Co-simulation result.

For example:
Bug Code: "#pragma HLS ARRAY_PARTITION variable=pool_window cyclic factor=2"
Correct Code: "#pragma HLS ARRAY_PARTITION variable=pool_window complete"
Where `pool_window` is being accessed by **pointer casting** to the index, where address mapping (pointer casting) is no longer consistent with index access with cyclic partitioning.

b. Resource Constraints:
Selecting a partition type that demands more resources than available (e.g., using complete partitioning for a very large array) can result in an APT error due to insufficient registers or memory resources.

Error Example 1
Error Code Description
When running this piece of code in C Simulation mode, I get the desired result, and 'Test passed !" is printed.
However, when I tried to partition the input signal `inA` with dimention 1, the synthesis failed and got the source synthesis failed: `ERROR: [HLS 200-1715] Encountered problem during source synthesis`.

Error Code
```CPP
#include "cpp_ap_int_arith.h"
```

```
void cpp_ap_int_arith(dinA_t    inA, dinB_t    inB, dinC_t    inC, dinD_t    inD,
                      dout1_t *out1, dout2_t *out2, dout3_t *out3, dout4_t *out4
   ) {

      #pragma HLS array_partition variable=inA type=complete dim=1

      *out1 = inA * inB;
      *out2 = inB + inA;
      *out3 = inC / inA;
      *out4 = inD % inA;

}
```

Error Reason and Fix
* Reason: The insertion of `#pragma HLS array_partition` is not applicable to input of a function/module due to missing proper buffering/interface delcartion.

* Fix: Remove the line `#pragma HLS array_partition variable=inA type=complete dim=1`
Error-Free Code
```CPP
#include "cpp_ap_int_arith.h"

void cpp_ap_int_arith(dinA_t    inA, dinB_t    inB, dinC_t    inC, dinD_t    inD,
                      dout1_t *out1, dout2_t *out2, dout3_t *out3, dout4_t *out4
   ) {


      *out1 = inA * inB;
      *out2 = inB + inA;
      *out3 = inC / inA;
      *out4 = inD % inA;

}
```

Error Example 2
Error Code Description
When running this piece of code in C Simulation mode, I get the desired result, and 'Test passed !" is printed. Also, C Synthesis is passed and RTL is successfully generated. However, when running it in C/RTL Cosimulation mode, the RTL result differs from C TB result and I got 'Test failed !" gets printed instead. In addition, I got tons of Warnings during C syntesis reporting

`pointer_cast_native/A` related ports has no fan-in and fan-out and be set to 0, for example:
```log
WARNING: [RTGEN 206-101] Setting dangling out port 'pointer_cast_native/A_0_d0' to 0.
WARNING: [RTGEN 206-101] Port 'pointer_cast_native/A_0_q0' has no fanin or fanout and is left dangling.
                    Please use C simulation to confirm this function argument can be read from or written to.
```
What can be causing the incorrect result in cosimulation mode?

Error Code
```CPP
data_t pointer_cast_native (data_t index, data_t A[N]) {
   dint_t* ptr;
   data_t i =0, result = 0;
   ptr = (dint_t*)(&A[index]);

   #pragma HLS ARRAY_PARTITION variable=A type=cyclic dim=1 factor=2
   for (i = 0; i < 4*(N/10); ++i) {
      result += *ptr;
      ptr+=1;
   }
   return result;
}
```

Error Reason and Fix
* Reason: This function is doing pointer casting with index of an array, which strongly relied on the actual address indexing of A. The line `ptr = (dint_t*)(&A[index])`; casts a pointer to a different data type. This can lead to alignment issues because the memory layout expected by dint_t might not match the actual layout of A. As the buffer is partioned in cyclic way, consecutive elements of A are placed in different memory banks. If `ptr` performs unaligned accesses or accesses that span multiple banks, it can cause inconsistent behavior.

* Fix: Change to `complete` type of partitioning (does not affect index/pointer relationship) or remove `#pragma HLS ARRAY_PARTITION variable=A type=cyclic dim=1 factor=2`.
Error-Free Code
```CPP
#include "pointer_cast_native.h"

data_t pointer_cast_native (data_t index,    data_t A[N]) {
   dint_t* ptr;
```

```
    data_t i =0, result = 0;
    ptr = (dint_t*)(&A[index]);

#pragma HLS ARRAY_PARTITION variable=A type=complete
    for (i = 0; i < 4*(N/10); ++i) {
        result += *ptr;
        ptr+=1;
    }
    return result;
}
```

Error Type: DFP
Error Related Structure: #pragma, dataflow, structure, function, pipeline
Error Related Pragma: '#pragma HLS dataflow'

Error Type Description

Basic Knowledge
DFP (Data Flow Pragma) error refers to incorrect placement of #pragma HLS dataflow. Unlike #pragma HLS pipeline that focuses on pipelining inner operations of a loop inside a function, #pragma HLS dataflow aims to schedule function-level pipelining between functions in a high-level module.

* Supported Code style in UG 1399:

1. Type 1: dataflow between functions

The canonical form for a function where sub-functions are not inlined. These subfunctions can themselves be dataflow in function regions or dataflow inside loop regions. Note also that variable initialization (including those performed automatically by constructors) or passing expressions by value to processes are not part of canonical form. For standard data types that have default constructors that cannot be redefined (For example: std::complex), this can be achieved by using no_ctor (For example: std::complex<float> arr[SIZE] __attribute__((no_ctor));). Vitis HLS does its best to implement the resulting dataflow but, if the code is not in canonical form, you should always check the GUI dataflow viewer and the cosimulation timeline trace to ensure that the dataflow happens as expected and the achieved performance is as expected.

Example for a correct dataflow pragma:
```CPP
void dataflow(Input0, Input1, Output0, Output1)
{
```

```
  #pragma HLS dataflow
  UserDataType C0, C1, C2; // UserDataType can be scalars or arrays
  func1(Input0, Input1, C0, C1); // read Input0, read Input1, write C0,
write C1
  func2(C0, C1, C2); // read C0, read C1, write C2
  func3(C2, Output0, Output1); // read C2, write Output0, write Output1
}
```

2. Type 2: dataflow within a loop with only functions

Dataflow inside a loop body enclosed in a function without any other code but the loop. For the for loop (where no function inside is inlined), the integral loop variable should have:
a. The initial value is declared in the loop header and set to 0.
b. The loop bound is a non-negative numerical constant or scalar argument of the function that encloses the loop.
c. Increment by 1.
d. Dataflow pragma needs to be inside the loop as shown below

```CPP
void dataflow(Input0, Input1, Output0, Output1)
{
  for (int i = 0; i < N; i++)
  {
  #pragma HLS dataflow
  UserDataType C0, C1, C2; // UserDataType can be scalars or
arrays
  func1(Input0, Input1, C0, C1); // read Input0, read Input1,
write C0, write C1
  func2(C0, C1, C2); // read C0, read C1, write C2
  func3(C2, Output0, Output1); // read C2, write Output0, write
Output1
  }
}
```

Error Knowledge
The DATAFLOW optimization allows functions and loops to overlap in their operation, increasing the concurrency of the RTL implementation. However, there are specific dos and don'ts to consider for writing a correct code with '#pragma HLS dataflow'. When you introduce bugs/errors into the correct code, you should take reverse considerations.
* Dos for a CORRECT insertion:

1. Place #pragma HLS dataflow at the beginning of the intended parallel region or just before parallel function calls.

2. Ensure data flows unidirectionally through the design from one task to the next.
3. Use it for function-level pipelining between functions in a high-level module.

* Don'ts for a CORRECT insertion:

1. Don't use it inside nested loops or functions that are not describing dataflow between functions.
2. Avoid single-producer-consumer violations.
3. Don't create feedback between tasks.
4. Don't use conditional execution of tasks within the dataflow region.
5. Avoid loops with multiple exit conditions in the dataflow region.

Error Example 1
Error Code Description
The code successfully passed CSim but failed during synthesis and the log reports `ERROR: [XFORM 203-313] Conflicting pipeline and dataflow directives found in function 'inner' (free_pipe_mult.cpp:44:9)
ERROR: [HLS 200-1715] Encountered problem during source synthesis
ERROR: [HLS 200-70] Pre-synthesis failed.`

Error Code
```CPP
#include "free_pipe_mult.h"
#include <iostream>

int accumulate(data_t A[]) {
#pragma HLS inline off

	data_t acc = 0.0;
	for (int i = 0; i < SZ; i++) {
		std::cout << "A: " << A[i] << std::endl;
		acc += A[i];
	}
	return acc;
}

void process(hls::stream<data_t>& strm_in, hls::stream<data_t>& strm_out) {
#pragma HLS inline off

	for (int i = 0; i < SZ; i++) {
		data_t tmp;
		tmp = strm_in.read();
		strm_out.write(tmp);
	}
```

```
}

void inner(data_t A[SZ], hls::stream<data_t>& stream_in, data_t* out) {
#pragma HLS DATAFLOW
#pragma HLS pipeline
#pragma HLS INTERFACE ap_fifo port = stream_in
	data_t regA[SZ];
#pragma HLS ARRAY_PARTITION variable = regA complete
	for (int i = 0; i < SZ; i++) {
		data_t tmp;
		tmp = stream_in.read();
		regA[i] = A[i] + tmp;
	}

	*out = accumulate(regA);
}

void free_pipe_mult(data_t A[SZ], hls::stream<data_t>& strm, data_t& out) {

#pragma HLS INTERFACE ap_fifo port = strm

	data_t B[SZ];

	for (int i = 0; i < SZ; i++)
		B[i] = A[i] + i;

	hls::stream<data_t> strm_out;
	process(strm, strm_out);
	inner(B, strm_out, &out);
}
```

Error Reason and Fix
* Reason: This code place `#pragma HLS DATAFLOW` inside a function that does not match the form as Type 1 and Type 2. The function inner only supports `#pragma HLS pipeline` as it is not describing the dataflow between functions.

* Fix: Move `#pragma HLS DATAFLOW` to the top module `free_pipe_mult` where decribes the dataflow between functions(modules), matching Type 1 scenario.

Error-Free Code
```CPP
#include "free_pipe_mult.h"
```

```
#include <iostream>

int accumulate(data_t A[]) {
#pragma HLS inline off

	data_t acc = 0.0;
	for (int i = 0; i < SZ; i++) {
		std::cout << "A: " << A[i] << std::endl;
		acc += A[i];
	}
	return acc;
}

void process(hls::stream<data_t>& strm_in, hls::stream<data_t>& strm_out) {
#pragma HLS inline off

	for (int i = 0; i < SZ; i++) {
		data_t tmp;
		tmp = strm_in.read();
		strm_out.write(tmp);
	}
}

void inner(data_t A[SZ], hls::stream<data_t>& stream_in, data_t* out) {

#pragma HLS pipeline
#pragma HLS INTERFACE ap_fifo port = stream_in
	data_t regA[SZ];
#pragma HLS ARRAY_PARTITION variable = regA complete
	for (int i = 0; i < SZ; i++) {
		data_t tmp;
		tmp = stream_in.read();
		regA[i] = A[i] + tmp;
	}

	*out = accumulate(regA);
}

void free_pipe_mult(data_t A[SZ], hls::stream<data_t>& strm, data_t& out) {
#pragma HLS DATAFLOW
#pragma HLS INTERFACE ap_fifo port = strm

	data_t B[SZ];
```

```
    for (int i = 0; i < SZ; i++)
        B[i] = A[i] + i;

    hls::stream<data_t> strm_out;
    process(strm, strm_out);
    inner(B, strm_out, &out);
}
```

Error Example 2

Error Reason: Single-producer-consumer violations

```c
void error_example(int in1, int in2, int *out) {
#pragma HLS DATAFLOW
    int tmp;
    func1(in1, &tmp); // Writing to tmp
    func2(in2, &tmp); // Another writing to tmp (violation)
    func3(tmp, out);    // Reading from tmp
}
```

Error Example 3

Error Reason: Feedback between tasks

```c
void error_example(int in1, int in2, int *out) {
#pragma HLS DATAFLOW
    int tmp1, tmp2;
    func1(in1, &tmp1);
    func2(tmp1, &tmp2);
    func3(tmp2, &tmp1); // Feedback from func3 to func1 (violation)
    func4(tmp1, out);
}
```

Error Example 4

Error Reason: Conditional execution of tasks

```c
void error_example(int in1, int in2, int cond, int *out) {
#pragma HLS DATAFLOW
    int tmp;
    func1(in1, &tmp);
    if (cond) {
        func2(tmp, out); // Conditional execution of func2 (violation)
```

```
    } else {
        func3(in2, out);
    }
}
```

Error Example 5
Error Reason: Loops with multiple exit conditions

```c
void error_example(int in1, int in2, int *out) {
#pragma HLS DATAFLOW
    int tmp;
    func1(in1, &tmp);
    for (int i = 0; i < 10; i++) {
        if (tmp > 0) break; // Multiple exit conditions (violation)
        func2(tmp, out);
        tmp--;
    }
}
```

Error Example 6
Error Reason: Reading from function inputs or writing to function outputs in the middle of the dataflow region

```c
void error_example(int in1, int in2, int *out1, int *out2) {
#pragma HLS DATAFLOW
    int tmp;
    func1(in1, &tmp);
    *out1 = in2; // Writing to function output in the middle (violation)
    func2(tmp, out2);
}
```

Guide for Identifying Error-Prone Code Regions

To identify error-prone code regions and insert `#pragma HLS DATAFLOW` to induce synthesis failures:

1. Look for functions or loops with multiple interconnected operations.
2. Identify data dependencies between these operations.
3. Check for the following patterns:
    a. Multiple functions or loops writing to the same variable.

b. Data flowing backwards between operations.
c. Conditional statements controlling the execution of functions or loops.
d. Loops with complex exit conditions or early breaks.
e. Operations that read function inputs or write to outputs in the middle of the function.

4. Insert `#pragma HLS DATAFLOW` just before the identified region.
5. If the region is a loop, ensure the loop doesn't conform to the canonical form (i.e., make sure it violates one of the conditions for proper dataflow in loops).

By following these steps, you can systematically identify code regions that are likely to cause dataflow optimization failures when the pragma is applied, effectively inserting DFP bugs into otherwise error-free code.

Error Type: DID
Error Related Structure: #pragma
Error Related Pragma: '#pragma HLS array_partition'

Error Type Description

Basic Knowledge
The Dim Incorrect Dimension (DID) error happens to a `#pragma HLS array_partition` that partitions an variable or an array that use fault dimension `dim=` for split. This value should be an integer ranging from 0 to <N> for an array that has <N> dimension. This only happens to the partition type of `block` and `cyclic` and WILL NOT AFFECT `complete` type of partitioning. This kind of error only applies to the code with '#pragma HLS array_partition' and related structures, based on your reasoning. If your code does not include this pragma, the DID error does not apply, and you should avoid inserting it unnecessarily.

DID error happens to fault partitioning the dimension for the target array (usually has dimension more than 1) that leads to data fault during the access.

Here are some basic illustration for the basic of array dimension for array partitioning :

* for array my_array[10][6][4]:
1. `#pragma HLS array_partition variable=my_array dim=3 factor=1 type=block`
This pragma splits the array by 3rd dimension, and we will get my_array_0[10][6], my_array_1[10][6], my_array_2[10][6], my_array_3[10][6]

2. `#pragma HLS array_partition variable=my_array dim=1 factor=1 type=block`
This pragma splits the array by 1st dimension, and we will get :
my_array_0[6][4]
my_array_1[6][4]
my_array_2[6][4]
my_array_3[6][4]

my_array_4[6][4]
my_array_5[6][4]
my_array_6[6][4]
my_array_7[6][4]
my_array_8[6][4]
my_array_9[6][4]

3. `#pragma HLS array_partition variable=my_array dim=0 factor=1 type=block`
This is equivalent to `#pragma HLS array_partition variable=my_array type=complete`, where all elements (10*6*4=240) in the array are partitioned into 240 independent registers.

Error Example 1
Error Code Description
This part will only show a fraction of loop that deal with different array access with '#pragma HLS array_partition'. In the given error code, the misuse of factor for array partitioning on a cyclic type leads to failure during Co-simulation.

Error Code
```CPP
int a[32];
int b[16][1024];

#pragma HLS array_partition variable=b dim=3 type=block factor=4

// a target to pipelined II=1
for(int i = 0; i < 16; i++) {
	#pragma HLS pipeline ii=1
	for(int j = 0; j < 1024; j++) {
		for (int k=0; k < 4, k++){
			b[i + k*4][j] = i + k*4 + j;
		}
	}
}
```
Error Reason and Fix
* Reason: The #pragma HLS array_partition directive incorrectly specifies dim=3 for a two-dimensional array b. Since b only has two dimensions (dim=1 and dim=2), specifying dim=3 is invalid.

* Fix: Understand the loop operation, change dim=1 to make it compatiable with the size of first index of b (16). If cannot understand, change the partition type to complete.

Error-Free Code

```CPP
int a[32];
int b[16][1024];

#pragma HLS array_partition variable=b dim=1 type=block factor=4

// a target to pipelined II=1
for(int i = 0; i < 16; i++) {
	#pragma HLS pipeline ii=1
	for(int j = 0; j < 1024; j++) {
		for (int k=0; k < 4, k++){
			b[i + k*4][j] = i + k*4 + j;
		}
	}
}
```

Error Type: FND
Error Related Structure: #pragma
Error Related Pragma: '#pragma HLS array_partition'

Error Type Description

The Factor Not Divisible (FND) error happens to fault partition factor on the target array (usually not in complete partitioning style) with '#pragma HLS array_partition' that leads to wrong data access or/and partition layout. This kind of error only applies to the code with '#pragma HLS array_partition' and related structures, based on your reasoning. If your code does not include this pragma, the FND error does not apply, and you should avoid inserting it unnecessarily.

Here are some basic example illustration for the factor on array dimension for array partitioning :
* for `array my_array[10][6][4]`:

1. `#pragma HLS array_partition variable=my_array dim=3 factor=1 type=block`
This pragma splits the array by 3rd dimension, and we will get my_array_0[10][6], my_array_1[10][6], my_array_2[10][6], my_array_3[10][6]

2. `#pragma HLS array_partition variable=my_array dim=3 factor=2 type=block`
This pragma splits the array by 3rd dimension, and factor of 2, and we will get my_array_0[10][6][2], my_array_1[10][6][2]

Error Example 1

Error Code Description
This part will only show a fraction of loop that deal with different array access with '#pragma HLS array_partition'. In the given error code, the misuse of factor for array partitioning on a cyclic type leads to failure during Co-simulation.

Error Code
```CPP
int a[32];

int b[16][1024];

// a target to pipelined II=1
#pragma HLS array_partition variable = a cyclic factor = 3
for(int i = 0; i < 8; i++) {
	#pragma HLS pipeline ii = 1
	for(int j = 0; j < 4; j++) {
		a[i*4 + j] = i;
	}
}
```
Error Reason and Fix
* Reason: This function tries to write array `a` in a pipelined manner with pipelined with 4 elements at the same time with 8 pipeline operations. This requires partitioning `a` by dimension of 4 to be compatiable with the cyclic type access of `a[i*4 + j] = i;`. Also, current factor 3 is not divisable for the size of 32 for array `a`, which leads to synthesis error.

* Fix: Understand the loop operation, and it seems to write all elemets in `a` with a 4-block, each block contains 8 element pattern. Change factor to 4, to make the loop compatiable with the target pipeline II=1. If cannot understand, change the partition type to complete.

Error-Free Code
```CPP
int a[32];

int b[16][1024];

// a target to pipelined II=1
#pragma HLS array_partition variable = a cyclic factor = 4
//#pragma HLS array_partition variable = a complete // if you don't understand the loop
for(int i = 0; i < 8; i++) {
	#pragma HLS pipeline ii = 1
	for(int j = 0; j < 4; j++) {
		a[i*4 + j] = i;
	}
```

```
}
```

Error Example 2
Error Code Description
This part will only show a fraction of loop that deal with different array access with '#pragma HLS array_partition'. In the given error code, the misuse of factor for array partitioning on a block type leads to failure during Co-simulation.

Error Code
```CPP
int a[32];

int b[16][1024];

#pragma HLS array_partition variable = a block factor = 3
	for(int i = 0; i < 8; i++) {
	#pragma HLS pipeline ii = 1
		for(int j = 0; j < 4; j++) {
			a[i + j*8] = i;
		}
}
```
Error Reason and Fix
* Reason: This function tries to write array `a` in a pipelined manner with pipelined with 4 elements at the same time with 8 pipeline operations, with a sequential, block manner. This requires partitioning `a` by dimension of 4 to be compatiable with the access of `a[i + j*8]`, which refers to a block type partitioning. Also, current factor 3 is not divisable for the size of 32 for array `a`, which leads to synthesis error.

* Fix: Understand the loop operation, and it seems to write all elemets in `a` with a 4-block, each block contains 8 element pattern. Change factor to 4, to make the loop compatiable with the target pipeline II=1. If cannot understand, change the partition type to complete.

Error-Free Code
```CPP
int a[32];

int b[16][1024];

#pragma HLS array_partition variable = a block factor = 4
//#pragma HLS array_partition variable = a complete // if you don't understand the loop
	for(int i = 0; i < 8; i++) {
```

```
    #pragma HLS pipeline ii = 1
        for(int j = 0; j < 4; j++) {
            a[i + j*8] = i;
        }
}
```

```md
# Error Type: MLP
## Error Related Structure: #pragma, pipeline, loop, unroll, nested loops, dataflow
### Error Related Pragma: '#pragma HLS PIPELINE'

## Error Type Description
### Basic Knowledge
MLP error means (Multi-Layer Nested Loop with Pipeline pragma). The `#pragma HLS PIPELINE` directive is used to enable pipelining in loops or functions, allowing new iterations to start before the previous ones have completed. This can significantly improve the throughput of the design by exploiting parallelism. However, when pipelining nested loops, care must be taken to avoid introducing errors or suboptimal results.

According to UG1399, when a loop or function is pipelined, Vitis HLS automatically unrolls all loops in the hierarchy below it. If there is a loop with variable bounds in this hierarchy, it prevents pipelining. To resolve this issue, the loop bounds should be made fixed, and conditional execution should be used inside the loop body.

### Bug Insertion Knowledge
To insert an MLP (Multi-Layer Nested Loop with Pipeline pragma) bug into an HLS code, follow these steps:

1. **Identify Nested Loop Structure**: Look for a code region with nested loops, where the outer loop iterates over a certain number of iterations, and the inner loop(s) perform computations such as data manipulations, additions, etc.

2. **Insert Pipeline Pragma at Multiple Levels**: If there is already a `#pragma HLS PIPELINE` in the loop structure, try to apply extra `#pragma HLS PIPELINE` directive at multiple levels of the nested loop hierarchy, such as the outer loop, inner loop, or both. If there is `#pragma HLS DATAFLOW` outside the loop, please note the difference between DFP error (Data Flow Pragma).

3. **Check for Variable Loop Bounds**: If any of the nested loops have multiple PIPELINE pragmas, it may prevent pipelining during synthesis and introduce the MLP bug.

4. **Exit Condition**: If the nested loop structure has fixed loop bounds at all levels, and there are no variable bounds, the code may not be suitable for introducing the MLP bug. In such cases,
```

exit the bug insertion process.

Bug Example 1
Error Code Description
This code performs a matrix multiplication operation using nested loops. The outer loop iterates over the rows of the output matrix, and the inner loops compute the dot product of the corresponding row from the first input matrix and the columns from the second input matrix.

Error Code
```c++
#include "ap_int.h"

#define SIZE 16

void matmul(int A[SIZE][SIZE], int B[SIZE][SIZE], int C[SIZE][SIZE]) {
#pragma HLS PIPELINE II=1
    for (int i = 0; i < SIZE; i++) {
#pragma HLS PIPELINE II=1
        for (int j = 0; j < SIZE; j++) {
            int sum = 0;
#pragma HLS PIPELINE II=1
            for (int k = 0; k < SIZE; k++) {
                sum += A[i][k] * B[k][j];
            }
            C[i][j] = sum;
        }
    }
}
```

Error Reason
In this code, the `#pragma HLS PIPELINE` directive is applied at multiple levels of the nested loop hierarchy, including the outermost loop, the middle loop, and the innermost loop. However, according to UG1399, when a loop or function is pipelined, Vitis HLS automatically unrolls all loops in the hierarchy below it.

In this case, pipelining the outermost loop will cause the middle and innermost loops to be unrolled, leading to a significant increase in hardware resources and potentially preventing pipelining due to resource constraints or data dependencies.

Similarly, pipelining the middle loop will cause the innermost loop to be unrolled, which may also lead to resource issues or data dependencies that prevent pipelining.

To resolve this issue, we should pipeline only the innermost loop, as it typically provides the best

balance between area and performance, and most important, follow the syntax rules for `#pragma HLS PIPELINE`.

Error Fix
1. Identify the issue:
    a. Multiple #pragma HLS PIPELINE directives are used across nested loops.
    b. This can lead to excessive resource usage and potential synthesis errors.

2. Remove unnecessary pipeline directives:
    a. Delete the pipeline directives from the outer and middle loops.
    b. Keep only the innermost pipeline directive.

3. Optimize the innermost loop:
    Ensure the innermost loop is pipelined with II=1 for maximum performance.

Correct Code
```c++
#include "ap_int.h"

#define SIZE 16

void matmul(int A[SIZE][SIZE], int B[SIZE][SIZE], int C[SIZE][SIZE]) {
    for (int i = 0; i < SIZE; i++) {
        for (int j = 0; j < SIZE; j++) {
            int sum = 0;
#pragma HLS PIPELINE II=1
            for (int k = 0; k < SIZE; k++) {
                sum += A[i][k] * B[k][j];
            }
            C[i][j] = sum;
        }
    }
}
```

Bug Example 2
Error Code Description
This code performs a convolution operation on a 2D input image using nested loops. The outer loops iterate over the output image dimensions, and the inner loops compute the convolution sum for each output pixel using a fixed-size kernel.

Error Code
```c++
```

```
#include "ap_int.h"

#define IMG_HEIGHT 480
#define IMG_WIDTH 640
#define KERNEL_SIZE 5

void conv2d(int input[IMG_HEIGHT][IMG_WIDTH], int kernel[KERNEL_SIZE][KERNEL_SIZE], int output[IMG_HEIGHT][IMG_WIDTH]) {
#pragma HLS PIPELINE II=1
    for (int y = 0; y < IMG_HEIGHT; y++) {
#pragma HLS PIPELINE II=1
        for (int x = 0; x < IMG_WIDTH; x++) {
            int sum = 0;
            for (int i = 0; i < KERNEL_SIZE; i++) {
#pragma HLS PIPELINE II=1
                for (int j = 0; j < KERNEL_SIZE; j++) {
                    sum += input[y + i - KERNEL_SIZE / 2][x + j - KERNEL_SIZE / 2] * kernel[i][j];
                }
            }
            output[y][x] = sum;
        }
    }
}
```

Error Reason

In this code, the `#pragma HLS PIPELINE` directive is applied at multiple levels of the nested loop hierarchy, including the outermost loop, the second loop, and the innermost loop. As mentioned earlier, pipelining the outermost loop will cause all inner loops to be unrolled, leading to a significant increase in hardware resources and potential resource constraints or data dependencies that prevent pipelining.

Additionally, pipelining the second loop will cause the innermost loop to be unrolled, which may also lead to resource issues or data dependencies that prevent pipelining.

To resolve this issue, we can pipeline the innermost loop and apply loop unrolling or flattening techniques to the outer loops, if necessary.

Error Fix

1. Identify the issue:
    a. Multiple #pragma HLS PIPELINE directives are used across nested loops.
    b. This can lead to excessive resource usage and potential synthesis errors.

2. Remove unnecessary pipeline directives:
    a. Delete the pipeline directives from the outer loops.
    b. Keep only the innermost pipeline directive.

3. Optimize the loop structure:
    a. Consider partial loop unrolling for the kernel loops.
    b. Use array partitioning for the kernel array to improve memory access.

4. Implement tiling to improve data reuse:
    a. Introduce tiling for the input image to process blocks at a time.

Correct Code
```c++
#include "ap_int.h"

#define IMG_HEIGHT 480
#define IMG_WIDTH 640
#define KERNEL_SIZE 5

void conv2d(int input[IMG_HEIGHT][IMG_WIDTH], int kernel[KERNEL_SIZE][KERNEL_SIZE], int output[IMG_HEIGHT][IMG_WIDTH]) {
#pragma HLS ARRAY_PARTITION variable=kernel complete dim=0
    for (int y = 0; y < IMG_HEIGHT; y++) {
#pragma HLS PIPELINE II=1
        for (int x = 0; x < IMG_WIDTH; x++) {
            int sum = 0;
#pragma HLS UNROLL
            for (int i = 0; i < KERNEL_SIZE; i++) {
                for (int j = 0; j < KERNEL_SIZE; j++) {
#pragma HLS PIPELINE II=1
                    sum += input[y + i - KERNEL_SIZE / 2][x + j - KERNEL_SIZE / 2] * kernel[i][j];
                }
            }
            output[y][x] = sum;
        }
    }
}
```

Error Type: OOB
Error Related Type: Memory Access, Array Acces, Index, Loops

Error Type Description

OOB means out-of-bound error. For HLS-C programming, such error may not be obvious due to C/C++ coding style does not have strictions on the size and width. The C cimulation may pass the testbench, even the code is successfully synthesized into RTL. This error may happen during the operation, access, declaration for types, index. The out-of-bound may happen for both input and output when handling with certain array, memory, even interface with inproper size or/and index. For inputs, the input data may have a larger size than the target array with smaller size. For outputs, the output data width may have larger width than the source memory with smaller width. This rule is also applied to the index access to an array.

This error originates from the difference in memory and data type management between software environments and hardware synthesis. In software, compilers automatically calculate the size of arrays. However, in hardware, the size must be explicitly defined, as out-of-bounds access can lead to undefined behavior after synthesis to RTL. This discrepancy can result in successful C simulation but failures during C/RTL co-simulation.

Error Example 1

Error Code Description

When running this piece of code in C Simulation mode, I get the desired result, and 'Test passed !" is printed. However, when running it in C/RTL Cosimulation mode, 'Test failed !" gets printed instead. What can be causing the incorrect result in cosimulation mode?

Error Code

```CPP
void runAutomata(char stream[], int stream_size, bool *match)
{
	#pragma HLS interface s_axilite port=stream
	#pragma HLS interface s_axilite port=match
	*match = kernel1(stream, stream_size);
}

int main()
{
	char stream[] = "abc";
	bool match;
	runAutomata(stream, 3, &match);
	if (match == true)
		printf("Test passed !\n");
	else
		printf("Test failed !\n");

	return 0;
}
```

```

### Error Reason and Fix
* Reason: Synthesized RTL cannot well detect the size of array without proper size decleration. In C simulation, the code run as software so `stream[]` has been automatically allocated with correct size. In C/RTL co simulation, as the array is has no index, nor it is static, the compiler failed to allocate correct size for the array, hence induce errors.

* Fix: Set peoper size for `char stream[]`
### Error-Free Code
```CPP
void runAutomata(char stream[4], int stream_size, bool *match)
{
	#pragma HLS interface s_axilite port=stream
	#pragma HLS interface s_axilite port=match
	*match = kernel1(stream, stream_size);
}

int main()
{
	char stream[4] = "abc";
	bool match;
	runAutomata(stream, 3, &match);
	if (match == true)
		printf("Test passed !\n");
	else
		printf("Test failed !\n");

	return 0;
}
```


Error Example 2
Error Code Description
When running this piece of code in C Simulation mode, I get the desired result, and 'Test passed !" is printed. However, when running it in C/RTL Cosimulation mode, 'Test failed !" gets printed instead. What can be causing the incorrect result in cosimulation mode?

Error Code
```CPP
#include "ap_fixed.h"
```
```

```
#include "ap_int.h"

#ifndef FIR_H_
#define FIR_H_

const unsigned char N=128;

typedef ap_int<8>   coef_t;
typedef ap_int<16>data_t;
typedef ap_int<16>acc_t;

void fir (
   data_t *y,
   data_t x
   );

#endif

#include "fir.h"

void fir (

   data_t *y,
   data_t x
   )
{
      static coef_t c[N] = {10, 11, 11, 8, 3, -3, -8, -11, -11, -10, -10, -10, -10, -10, -10, -10, -10, -10,
-10, -10, -10, -10, -10, -10, -10, -10, -10, -10, -10, -10, -10, -10, -10, -11, -11, -8, -3, 3, 8, 11, 11,
10, 10, 10, 10, 10, 10, 10, 10, 11, 11, 8, 3, -3, -8, -11, -11, -10, -10, -10, -10, -10, -10, -10, -10, -11,
-11, -8, -3, 3, 8, 11, 11, 10, 10, 10, 10, 10, 10, 10, 10, 11, 11, 8, 3, -3, -8, -11, -11, -10, -10, -10, -10,
-10, -10, -10, -10, -11, -11, -8, -3, 3, 8, 11, 11, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10,
10, 10, 10, 10, 10, 10, 10, 10, 10};

      char i;
      static data_t Mem[N]={0};
#pragma HLS ARRAY_PARTITION variable=Mem complete
      acc_t result;

      result = 0;

#pragma HLS PIPELINE
      for (i=N-1; i>0; i--){
#pragma HLS UNROLL factor=4
            Mem[i] = Mem[i-1];
```

```
            result += Mem[i] * c[i];
        }

        Mem[0] = x;
        result += Mem[0] * c[i];
        *y = result;
}

```

Error Reason and Fix
* Reason: Incorrect data with definiton for input and output data. As the range for input and output data is different. In this case, the assigned datatype from ap_int library should be able to handle positive and negative number as input and output. When it the detection of the range to determine the data with is difficult, fallback to C/C++ data types to ensure the design could pass the co-simulation. If the data width could be analyzed from the data flow, apply appropriate data with.

* Fix: Fallback the datatype of input to `int` or large C/C++ data type, or extend the data width based on the data range of input and output. Change datatype in header and leave top module unchanged.
Error-Free Code
```CPP
#include "ap_fixed.h"
#include "ap_int.h"

#ifndef FIR_H_
#define FIR_H_

const unsigned char N=128;

typedef ap_int<8>   coef_t;
typedef int       data_t;
typedef ap_int<16>acc_t;

void fir (
    data_t *y,
    data_t x
    );

#endif
//*** following are unchanged
```

//
Error Example 3
Error Code Description
The code passed compilation but gets get error in both C simulation and Co-simulation when comparing with my pre-defined golden samples. Please help me to find possible errors I have made.

Error Code
```CPP
void BoundIDctMatrix (int *matrix, int Bound)
{
   int *mptr;

   for (mptr = matrix; mptr <= matrix + DCTSIZE2; mptr++)
      {
         if (*mptr < 0)
      {
         *mptr = 0;
      }
         else if (*mptr > Bound)
      {
         *mptr = Bound;
      }
      }
}
```

Error Reason and Fix
* Reason: incorrectly set the boundray for the loop iteration with `mptr` by `<=` rather than `<`, inducing an out-of-bound access

* Fix: Should refer to the specific function. In this case, the `mptr` has strong numerical relationship with `matrix + DCTSIZE2` for detecting wheteher the pointer is out of bound. By checking the related function, change `mptr <= matrix + DCTSIZE2` to `mptr < matrix + DCTSIZE2` solved the issue.

Error-Free Code
```CPP
void BoundIDctMatrix (int *matrix, int Bound)
{
   int *mptr;
```

```
    for (mptr = matrix; mptr < matrix + DCTSIZE2; mptr++)
        {
            if (*mptr < 0)
        {
            *mptr = 0;
        }
            else if (*mptr > Bound)
        {
            *mptr = Bound;
        }
        }
}
```

Error Type: RAMB (Random Access Memory with Burst mode)
Error Related Structure: #pragma, AXI, burst mode, memory
Error Related Pragma: '#pragma HLS INTERFACE'

Error Type Description
Basic Knowledge
The AXI4 master interface (m_axi) in Vitis HLS allows for burst-type data transfers, which can significantly improve memory access efficiency. To enable burst transfers, the '#pragma HLS INTERFACE m_axi' directive is used. Burst mode requires sequential, predictable memory access patterns to function correctly. This error may not always induce error at Csim stage, which means the code behavior is still correct but not truly compatiable with burst mode data transfer in AXI. However, developer can observe poor performance during AXI read/write process in co-simulation process.

Bug Insert Knowledge
RAMB errors occur when the memory access pattern in the code is incompatible with burst transfer requirements. To insert this bug:
1. Use non-sequential memory accesses
2. Implement random or unpredictable access patterns
3. Mix read and write operations in an interleaved manner
4. Access memory locations that overlap or have gaps between them

These patterns violate the preconditions for burst transfers as outlined in UG1399, Chapter 18.

Bug Example 1
Error code description
This function attempts to process an array by accessing elements in a non-sequential manner, which is incompatible with burst mode transfers.

Error code

```cpp
void process_array(int *input, int *output, int size) {
	#pragma HLS INTERFACE m_axi port=input
	#pragma HLS INTERFACE m_axi port=output

	for (int i = 0; i < size; i++) {
		output[i] = input[size - i - 1] + input[i];
	}
}
```

Error Reason

This code violates burst transfer requirements because:

1. The input array is accessed in reverse order (input[size - i - 1]), which is not a monotonically increasing access pattern.
2. The input array is also accessed in forward order (input[i]), creating overlapping accesses.
3. These non-sequential, overlapping accesses prevent the HLS tool from inferring efficient burst transfers.

Error Fix

1. We first read the entire input array into a local buffer using a sequential access pattern, allowing for burst reads.
2. We then process the data using the local buffer, which doesn't affect burst transfers.
3. Finally, we write the output in a sequential manner, enabling burst writes.

This approach ensures that all memory accesses to the AXI interfaces are sequential and predictable, allowing the HLS tool to infer and implement efficient burst transfers.

Correct Code

```cpp
void process_array(int *input, int *output, int size) {
	#pragma HLS INTERFACE m_axi port=input
	#pragma HLS INTERFACE m_axi port=output

	// Read input into local buffer
	int local_input[1024];
	for (int i = 0; i < size; i++) {
		#pragma HLS PIPELINE
		local_input[i] = input[i];
	}

	// Process data
	for (int i = 0; i < size; i++) {
		#pragma HLS PIPELINE
		output[i] = local_input[size - i - 1] + local_input[i];
```

```
    }
}
```

Bug Example 2
Error code description
This function attempts to process an array by accessing elements with a variable stride, which is incompatible with burst mode transfers.

Error code
```cpp
void process_with_stride(int *input, int *output, int size, int stride) {
    #pragma HLS INTERFACE m_axi port=input
    #pragma HLS INTERFACE m_axi port=output

    for (int i = 0; i < size; i++) {
        output[i] = input[i * stride];
    }
}
```
Error Reason
This code violates burst transfer requirements because:

1. The input array is accessed with a variable stride (`input[i * stride]`), which may not result in consecutive memory accesses.
2. The stride value is unknown at compile-time, making it impossible for the HLS tool to guarantee monotonically increasing and consecutive accesses.
3. These non-consecutive accesses prevent the HLS tool from inferring efficient burst transfers.

Error Fix
1. Identify the issue: The function uses a variable stride to access input elements, which prevents efficient burst transfers.
2. Analyze the problem: Variable stride access creates non-consecutive memory accesses, making it impossible for the HLS tool to infer burst transfers.
3. Consider solutions:
    a. Use a local buffer to store input data in a consecutive manner.
    b. Separate the reading and processing stages.
4. Implement the solution:
    a. Create a local buffer to store input data.
    b. Read input data sequentially into the local buffer.
    c. Process data from the local buffer using the original stride.
5. Verify: Ensure that all AXI interface accesses are now sequential and predictable.

Correct Code
```CPP
void process_with_stride(int *input, int *output, int size, int stride) {
	#pragma HLS INTERFACE m_axi port=input
	#pragma HLS INTERFACE m_axi port=output

	// Read input into local buffer
	int local_input[1024];
	for (int i = 0; i < size * stride; i++) {
		#pragma HLS PIPELINE
		local_input[i] = input[i];
	}

	// Process data with stride
	for (int i = 0; i < size; i++) {
		#pragma HLS PIPELINE
		output[i] = local_input[i * stride];
	}
}
```

Bug Example 3
Error code description
Error code description
This function attempts to process two arrays simultaneously, which can interfere with burst mode transfers when both arrays are mapped to the same AXI port.

Error code
```cpp
void process_two_arrays(int *input1, int *input2, int *output, int size) {
	#pragma HLS INTERFACE m_axi port=input1
	#pragma HLS INTERFACE m_axi port=input2
	#pragma HLS INTERFACE m_axi port=output

	for (int i = 0; i < size; i++) {
		output[i] = input1[i] + input2[i];
	}
}
```
Error Reason
This code may violate burst transfer requirements because:

1. If `input1` and `input2` are mapped to the same M-AXI port, bursting will be done for only one array at a time.

2. The interleaved access pattern (reading from `input1`, then `input2`, then writing to output) can prevent efficient burst transfers.
3. These interleaved accesses across multiple arrays can make it difficult for the HLS tool to infer efficient burst transfers.

Error Fix
1. Identify the issue: The function processes two input arrays simultaneously, which can interfere with burst transfers if mapped to the same AXI port.
2. Analyze the problem: Interleaved access to multiple arrays prevents efficient burst transfers and makes it difficult for the HLS tool to optimize.
3. Consider solutions:
    a. Use local buffers for each input array.
    b. Separate the reading, processing, and writing stages.
4. Implement the solution:
    a. Create local buffers for both input arrays.
    b. Read each input array sequentially into its respective local buffer.
    c. Process data using the local buffers.
    d. Write output sequentially.
5. Verify: Ensure that all AXI interface accesses are now sequential and independent for each array.

Correct Code
```cpp
void process_two_arrays(int *input1, int *input2, int *output, int size) {
    #pragma HLS INTERFACE m_axi port=input1
    #pragma HLS INTERFACE m_axi port=input2
    #pragma HLS INTERFACE m_axi port=output

    int local_input1[1024], local_input2[1024];

    // Read input1 into local buffer
    for (int i = 0; i < size; i++) {
        #pragma HLS PIPELINE
        local_input1[i] = input1[i];
    }

    // Read input2 into local buffer
    for (int i = 0; i < size; i++) {
        #pragma HLS PIPELINE
        local_input2[i] = input2[i];
    }

    // Process data and write output
    for (int i = 0; i < size; i++) {
```

```
            #pragma HLS PIPELINE
            output[i] = local_input1[i] + local_input2[i];
        }
}
```

Error Type: SMA (Scalar to Memory Array)
Error Related Structure: #pragma, AXI, array, memory
Error Related Pragma: '#pragma HLS INTERFACE'

Error Type Description
Basic Knowledge
The `#pragma HLS INTERFACE` is used to specify the hardware interface for function arguments and global variables. For scalar values, the default interface is typically ap_none for inputs and ap_vld for outputs in the Vivado IP flow. In the Vitis Kernel flow, scalar arguments are implemented using the AXI4-Lite (s_axilite) interface. Arrays, on the other hand, are typically implemented using interfaces like ap_memory, bram, or m_axi, which are designed to handle multiple data elements.

For scalar values:

* In the Vivado IP flow:
1. `ap_none`: Inputs typically use the interface by default.
2. `ap_vld`: Outputs typically use the interface by default.
* In the Vitis Kernel flow:
1. `s_axilite`: Scalar arguments passed by value are implemented using the AXI4-Lite interface.
2. `s_axilite`: Pointers to scalar values are also implemented using the AXI4-Lite interface.

For arrays:
1. `ap_memory`: The default interface is typically .
2. Other common interfaces for arrays include:
    A. `bram`: Similar to ap_memory, but appears as a single grouped port in IP integrator.
    B. `m_axi`: Used in the Vitis Kernel flow for arrays and pointers to arrays.
    C. `ap_fifo`: Can be used if the array is accessed sequentially.

Bug Insert Knowledge
To identify and insert an SMA bug, follow these steps:

1. Locate the functions with '#pragma HLS INTERFACE' in the HLS design. If the design does not have this kind of pragma, stop and do nothing.
2. Identify scalar input arguments in the function declaration. If there is no scalar values as input or/and output, stop and do nothing.
3. Find the corresponding `#pragma HLS INTERFACE` statements for these scalar inputs. If there

is no scalar definition does with '#pragma HLS INTERFACE' pragma, stop and do nothing.
4. Choose an array interface (e.g., bram, ap_memory, or m_axi) to replace the correct scalar interface.
5. Modify the pragma to use the chosen array interface for the scalar input.
6. Repeat steps 4-5 for other scalar inputs if desired.

Bug Example 1
Error code description
This example shows a simple function that takes a scalar input and returns a scalar output. The bug is introduced by incorrectly using the bram interface for the scalar input.

Error code
```cpp
int scalar_function(int input_scalar) {
#pragma HLS INTERFACE mode=bram port=input_scalar
    return input_scalar * 2;
}
```

Error Reason
The error in this code is that we're using the bram interface for a scalar value (input_scalar). The bram interface is designed for arrays and requires address signals, which are not applicable to scalar values. This will likely result in synthesis errors or incorrect hardware implementation.

To correct this, we should use an appropriate interface for scalar values, such as ap_none or s_axilite, depending on the target flow.

Error Fix
Let's fix this error step-by-step:

1. Identify the problem: The scalar value 'input_scalar' is incorrectly using a bram interface.
2. Understand the correct interface: Scalar values should use s_axilite, ap_none, or ap_vld interfaces.
3. Choose the appropriate interface: For the Vitis Kernel flow, s_axilite is suitable.
4. Modify the pragma: Change the interface mode from bram to s_axilite.
5. Add return interface: Include a pragma for the return value using s_axilite.
6. Verify the changes: Ensure all scalar values use appropriate interfaces.

Correct Code
```cpp
int scalar_function(int input_scalar) {
```

```
#pragma HLS INTERFACE mode=s_axilite port=input_scalar
#pragma HLS INTERFACE mode=s_axilite port=return
    return input_scalar * 2;
}
```

Bug Example 2
Error code description
This example demonstrates a function that takes multiple scalar inputs and incorrectly uses the m_axi interface for one of them.

Error code
```cpp
void multi_scalar_function(int a, int b, int c, int *result) {
#pragma HLS INTERFACE mode=s_axilite port=a
#pragma HLS INTERFACE mode=m_axi port=b
#pragma HLS INTERFACE mode=s_axilite port=c
#pragma HLS INTERFACE mode=s_axilite port=result
    *result = a + b + c;
}
```

Error Reason
The error in this code is the use of the m_axi interface for the scalar input 'b'. The m_axi interface is designed for arrays or pointers to access external memory, not for individual scalar values. This will likely cause synthesis issues or inefficient hardware implementation.

To correct this, we should use the s_axilite interface for all scalar inputs, which is appropriate for the Vitis Kernel flow.

Error Fix
Let's fix this error step-by-step:

1. Identify the problem: The scalar value 'b' is incorrectly using an m_axi interface.
2. Understand the correct interface: All scalar inputs should use the same interface type.
3. Choose the appropriate interface: s_axilite is suitable for all scalar inputs in this case.
4. Modify the pragma: Change the interface mode for 'b' from m_axi to s_axilite.
5. Verify consistency: Ensure all scalar inputs (a, b, c) use the s_axilite interface.
6. Add return interface: Include a pragma for the return value using s_axilite.
7. Review the changes: Confirm all interfaces are correctly specified for scalar values.

Correct Code
```cpp
```

```
void multi_scalar_function(int a, int b, int c, int *result) {
#pragma HLS INTERFACE mode=s_axilite port=a
#pragma HLS INTERFACE mode=s_axilite port=b
#pragma HLS INTERFACE mode=s_axilite port=c
#pragma HLS INTERFACE mode=s_axilite port=result
#pragma HLS INTERFACE mode=s_axilite port=return
    *result = a + b + c;
}
```


{
  "Errors": [
    {
      "Type": "Memory Errors",
      "Details": [
        {
          "error_type": "OOB",
          "description": "OOB means out-of-bound. However, for HLS-C programming, such error may not be obvious due to C/C++ coding style does not have restrictions on the size and width. The C simulation may pass the testbench, even the code is successfully synthesized into RTL. This error may happen during the operation, access, declaration for types, index. The out-of-bound may happen for both input and output when handling with certain array, memory, even interface with improper size or/and index. For inputs, the input data may have a larger size than the target array with smaller size. For outputs, the output data width may have larger width than the source memory with smaller width. This rule is also applied to the index access to an array.\n\nThis error originates from the difference in memory and data type management between software environments and hardware synthesis. In software, compilers automatically calculate the size of arrays. However, in hardware, the size must be explicitly defined, as out-of-bounds access can lead to undefined behavior after synthesis to RTL. This discrepancy can result in successful C simulation but failures during C/RTL co-simulation.",
          "error_examples": [
            {
              "example_number": 1,
              "error_code_description": "When running this piece of code in C Simulation mode, I get the desired result, and 'Test passed !\" is printed. However, when running it in C/RTL Cosimulation mode, 'Test failed !\" gets printed instead. What can be causing the incorrect result in cosimulation mode?",
              "error_code": "void runAutomata(char stream[], int stream_size, bool *match)\n{\n   #pragma HLS interface s_axilite port=stream\n   #pragma HLS interface s_axilite port=match\n   *match = kernel1(stream, stream_size);\n}\n \nint main()\n{\n\tchar stream[] = \"abc\";\n\tbool match;\n\trunAutomata(stream, 3, &match);\n\tif (match == true)\n\t\tprintf(\"Test passed !\\n\");\n\telse\n\t\tprintf(\"Test failed !\\n\");\n \n\treturn 0;\n}",
              "error_reason": "Synthesized RTL cannot well detect the size of array without proper size declaration. In C simulation, the code run as software so `stream[]` has been

automatically allocated with correct size. In C/RTL co simulation, as the array is has no index, nor it is static, the compiler failed to allocate correct size for the array, hence induce errors."
},
{
"example_number": 2,
"error_code_description": "When running this piece of code in C Simulation mode, I get the desired result, and 'Test passed !\" is printed. However, when running it in C/RTL Cosimulation mode, 'Test failed !\" gets printed instead. What can be causing the incorrect result in cosimulation mode?",
"error_code": "#include \"ap_fixed.h\"\n#include \"ap_int.h\"\n\n#ifndef FIR_H_\n#define FIR_H_\n\nconst unsigned char N=128;\n\ntypedef ap_int<8>\tcoef_t;\ntypedef ap_int<16>\tdata_t;\ntypedef ap_int<16>\tacc_t;\n\nvoid fir (\n data_t *y,\n data_t x\n);\n\n#endif\n\n#include \"fir.h\"\n\nvoid fir (\n\n data_t *y,\n data_t x\n)\n{\n\tstatic coef_t c[N] = {10, 11, 11, 8, 3, -3, -8, -11, -11, -10, -10, -10, -10, -10, -10, -10, -10, -10, -10, -10, -10, -10, -10, -10, -10, -10, -10, -10, -10, -10, -10, -10, -10, -11, -11, -8, -3, 3, 8, 11, 11, 10, 10, 10, 10, 10, 10, 10, 10, 11, 11, 8, 3, -3, -8, -11, -11, -10, -10, -10, -10, -10, -10, -10, -10, -10, -11, -11, -8, -3, 3, 8, 11, 11, 10, 10, 10, 10, 10, 10, 10, 10, 11, 11, 8, 3, -3, -8, -11, -11, -10, -10, -10, -10, -10, -10, -10, -10, -11, -11, -8, -3, 3, 8, 11, 11, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10};\n\n\tchar i;\n\tstatic data_t Mem[N]={0};\n#pragma HLS ARRAY_PARTITION variable=Mem complete\n\tacc_t result;\n\n\tresult = 0;\n\n#pragma HLS PIPELINE\n\tfor (i=N-1; i>0; i--){\n#pragma HLS UNROLL factor=4\n\t\tMem[i] = Mem[i-1];\n\t\tresult += Mem[i] * c[i];\n\t}\n\n\tMem[0] = x;\n\tresult += Mem[0] * c[i];\n\t*y = result;\n}",
"error_reason": "Incorrect data with definition for input and output data. As the range for input and output data is different. In this case, the assigned datatype from ap_int library should be able to handle positive and negative number as input and output. When it the detection of the range to determine the data with is difficult, fallback to C/C++ data types to ensure the design could pass the co-simulation. If the data width could be analyzed from the data flow, apply appropriate data with."
},
{
"example_number": 3,
"error_code_description": "The code passed compilation but gets get error in both C simulation and Co-simulation when comparing with my pre-defined golden samples. Please help me to find possible errors I have made.",
"error_code": "void BoundIDctMatrix (int *matrix, int Bound)\n{\n int *mptr;\n\n for (mptr = matrix; mptr <= matrix + DCTSIZE2; mptr++)\n {\n if (*mptr < 0)\n\t{\n\t *mptr = 0;\n\t}\n else if (*mptr > Bound)\n\t{\n\t *mptr = Bound;\n\t}\n }\n}",
"error_reason": "incorrectly set the boundary for the loop iteration with `mptr` by `<=` rather than `<`, inducing an out-of-bound access"
}
]
},

{
"error_type": "BUF",
"description": "Copying from the wrong half of a split buffer due to incorrect indexing.",
"error_examples": [
{
"example_number": 1,
"error_code_description": "Should iterate over 16, but DEPTH is used which might be different",
"error_code": "for(int co = 0; co < DEPTH; co+=16) { // Faulty Code\nfor(int co = 0; co < 16; co+=16) { // Original Code",
"error_reason": "The loop should iterate over a fixed size of 16, but incorrectly uses `DEPTH` which could be a different value. Correcting the iteration condition to a constant `16` ensures proper indexing."
},
{
"example_number": 2,
"error_code_description": "Copying from the wrong half of a split buffer due to incorrect indexing.",
"error_code": "b_copy[k][j] = a[k][j]; // Faulty Code\nb_copy[k][j] = a[k+DIM][j]; // Original Code",
"error_reason": "The faulty code incorrectly copies from the first half of the buffer `a` instead of the intended second half. Adjusting the index to `k+DIM` corrects the indexing to access the right half."
}
]
},
{
"error_type": "APT",
"description": "Misuse of the #pragma HLS array_partition directive.",
"error_examples": [
{
"example_number": 1,
"error_code_description": "Incorrect use of the #pragma HLS array_partition directive with type cyclic when it should be block.",
"error_code": "#pragma HLS ARRAY_PARTITION variable=weight_buf type=cyclic dim=2 factor=16 // Faulty Code\n#pragma HLS ARRAY_PARTITION variable=weight_buf dim=2 factor=16 // Original Code",
"error_reason": "The misuse of `type=cyclic` for the array partitioning directive. Removing the type specification defaults to the correct block partitioning."
},
{
"example_number": 2,
"error_code_description": "Incorrect array partition type specified for the

variable.",
"error_code": "#pragma HLS array_partition type=complete variable=x // Faulty Code\n#pragma HLS PIPELINE // Original Code",
"error_reason": "The faulty code incorrectly specifies partition type for a variable not intended for partitioning. Removing the incorrect directive fixes the issue."
}
]
},
{
"error_type": "FND",
"description": "Using a partition factor that does not evenly divide the array size.",
"error_examples": [
{
"example_number": 1,
"error_code_description": "Partition factor does not evenly divide the array size, leading to uneven partitioning.",
"error_code": "#pragma HLS ARRAY_PARTITION variable=bottom cyclic dim=1 factor=15 // Faulty Code\n#pragma HLS ARRAY_PARTITION variable=bottom cyclic dim=1 factor=16 // Original Code",
"error_reason": "The partition factor `15` does not evenly divide the array size, causing an incorrect partition. Changing the factor to `16` ensures even partitioning."
},
{
"example_number": 2,
"error_code_description": "Incorrect partition factor used for the array, affecting performance.",
"error_code": "#pragma HLS ARRAY_PARTITION variable=a cyclic factor=4 dim=2 // Faulty Code\n#pragma HLS INTERFACE bram port=a // Original Code",
"error_reason": "The use of an incorrect partition factor `4` could lead to suboptimal performance. Correcting the partition factor to match the array size or removing the partition directive ensures optimal resource usage."
}
]
},
{
"error_type": "DID",
"description": "Specifying a wrong dimension in the #pragma HLS array_partition.",
"error_examples": [
{
"example_number": 1,
"error_code_description": "Wrong dimension specified in array partition, leading to incorrect partitioning.",
"error_code": "#pragma HLS ARRAY_PARTITION variable=bottom cyclic dim=3 factor=16 // Faulty Code\n#pragma HLS ARRAY_PARTITION variable=bottom cyclic

dim=1 factor=16 // Original Code",
"error_reason": "Specifying the wrong dimension `3` for the array partition leads to incorrect behavior. Correcting the dimension to `1` ensures the correct partitioning strategy is applied."
},
{
"example_number": 2,
"error_code_description": "Incorrect dimension specified for array partition, leading to synthesis issues.",
"error_code": "#pragma HLS array_partition variable=accumulators dim=2 complete // Faulty Code\n#pragma HLS bind_storage variable=accumulators type=RAM_2P impl=LUTRAM // Original Code",
"error_reason": "The incorrect specification of dimension `2` for complete partitioning may lead to synthesis or functionality issues. Correcting the directive to align with the intended hardware implementation resolves the issue."
}
]
},
{
"error_type": "AMS",
"description": "Incorrectly mapping entire arrays to interfaces designed for scalar data types.",
"error_examples": [
{
"example_number": 1,
"error_code_description": "Entire array incorrectly mapped to an interface designed for scalar types.",
"error_code": "#pragma HLS INTERFACE ap_none port=a // Faulty Code\n#pragma HLS INTERFACE ap_ctrl_hs port=return // Original Code",
"error_reason": "Mapping an entire array to a scalar interface type `ap_none` leads to interface mismatches. Using the correct interface directive `ap_ctrl_hs` for scalar types ensures correct interface mapping."
},
{
"example_number": 2,
"error_code_description": "Array incorrectly interfaced, leading to potential synthesis or functionality issues.",
"error_code": "#pragma HLS INTERFACE port=weight_buf mode=ap_none // Faulty Code\n#pragma HLS ARRAY_PARTITION variable=weight_buf dim=1 factor=16 // Original Code",
"error_reason": "Incorrect interfacing of the array `weight_buf` as a scalar type. Correcting the interface directive to properly partition the array ensures correct synthesis and functionality."
}

```
            ]
        }
    ]
},
{
    "Type": "Initialization Errors",
    "Details": [
        {
            "error_type": "INIT",
            "description": "Using a variable before it has been assigned a value, which can lead to unpredictable behavior or hardware that does not behave as expected.",
            "error_examples": [
                {
                    "example_number": 1,
                    "oracle_code": "FIX_32_12 tmp[DEPTH];",
                    "buggy_code": "// FIX_32_12 tmp[DEPTH];"
                },
                {
                    "example_number": 2,
                    "oracle_code": "mat_type tmp = 0;",
                    "buggy_code": "mat_type tmp;"
                }
            ]
        },
        {
            "error_type": "ZERO",
            "description": "When assigning a value to a variable, mistakenly initialized the variable to zero.",
            "error_examples": [
                {
                    "example_number": 1,
                    "oracle_code": "tmp[DEPTH];",
                    "buggy_code": "tmp[DEPTH] = {0};"
                },
                {
                    "example_number": 2,
                    "oracle_code": "it=1; ft=2; ot=3; gt=4;",
                    "buggy_code": "it = 0; ft = 0; ot = 0; gt = 0;"
                }
            ]
        }
    ]
},
{
```

"Type": "Logic Errors",
"Details": [
{
"error_type": "INF",
"description": "Loops without proper exit conditions can prevent synthesis or lead to non-functional hardware.",
"error_examples": [
{
"example_number": 1,
"error_code_description": "Loop without a proper exit condition, leading to an infinite loop.",
"error_code": "for(int h = 1; h != 0; h++) // Faulty Code\nfor(int h = 1; h <= HEIGH-2; h++) // Original Code",
"error_reason": "The faulty loop condition `h != 0` can potentially create an infinite loop, preventing synthesis. Changing the condition to `h <= HEIGH-2` ensures a proper exit condition."
},
{
"example_number": 2,
"error_code_description": "Loop without a proper exit condition, leading to an infinite loop.",
"error_code": " SUM:for (i=0; ; i++) { \n accum += I[i];\n O[i] = accum;\n }",
"error_reason": "The faulty loop condition `for (i=0; ; i++)` can create an infinite loop."
}
]
},
{
"error_type": "MLU",
"description": "Incorrect unrolling of loops can cause synthesis failures or inefficient hardware.",
"error_examples": [
{
"example_number": 1,
"error_code_description": "Incorrect loop boundary conditions, potentially causing synthesis failures.",
"error_code": "for(int i = 0; i < 255; i++){} // Faulty Code\nfor(int i = 0; i < 256; i++){} // Original Code",
"error_reason": "The loop is unrolled with an incorrect boundary condition, affecting the expected iteration count. Correcting the condition to `i < 256` ensures accurate loop execution."
},
{

"example_number": 2,
"error_code_description": "Incorrect increment in loop unrolling, causing inefficient hardware.",
"error_code": "for(int co = 0; co < 16; co+=8) {} // Faulty Code\nfor(int co = 0; co < 16; co+=16) {} // Original Code",
"error_reason": "The loop increment is set to `co+=8` instead of `co+=16`, leading to a potential doubling of iterations. Correcting the increment to `co+=16` ensures proper loop unrolling."
}
]
},
{
"error_type": "DFP",
"description": "Misplacement of #pragma HLS dataflow can disrupt the intended order of operations.",
"error_examples": [
{
"example_number": 1,
"error_code_description": "Misplaced #pragma HLS dataflow affecting the flow of operations.",
"error_code": "#pragma HLS dataflow\n\t\t\tfor(int h = 1; h <= HEIGH-2; h++){} // Faulty Code\nfor(int h = 1; h <= HEIGH-2; h++){} // Original Code",
"error_reason": "The `#pragma HLS dataflow` is incorrectly placed before a loop, potentially disrupting the intended order. Removing or correctly positioning the pragma ensures the proper flow of operations."
},
{
"example_number": 2,
"error_code_description": "Incorrect replacement of pipeline pragma with dataflow, disrupting intended parallelism.",
"error_code": "#pragma HLS dataflow // Faulty Code\n#pragma HLS pipeline // Original Code",
"error_reason": "Replacing `#pragma HLS pipeline` with `#pragma HLS dataflow` could disrupt the intended parallel execution. Reverting to the original pipeline pragma ensures correct implementation."
}
]
},
{
"error_type": "ILNU",
"description": "Partially unrolled loops can cause unpredictable performance.",
"error_examples": [
{
"example_number": 1,

"error_code_description": "Incorrect unroll factor, leading to unpredictable loop performance.",
"error_code": "#pragma HLS unroll factor=4 // Faulty Code\n#pragma HLS unroll // Original Code",
"error_reason": "Specifying an unroll factor of `4` may not align with the loop's optimal execution strategy. Removing the factor or adjusting it to match the hardware capabilities ensures predictable performance."
}
]
},
{
"error_type": "MLP",
"description": "Incorrect or overly aggressive use of #pragma HLS pipeline can lead to resource conflicts.",
"error_examples": [
{
"example_number": 1,
"error_code_description": "Misuse of pipeline pragma, leading to potential resource conflicts.",
"error_code": "#pragma HLS pipeline // Faulty Code\n#pragma HLS dataflow // Original Code",
"error_reason": "Incorrectly applying `#pragma HLS pipeline` where `#pragma HLS dataflow` is intended can cause resource conflicts or inefficiencies. Correctly using the dataflow pragma ensures optimal resource utilization."
},
{
"example_number": 2,
"error_code_description": "Pipeline pragma incorrectly placed inside an if statement, causing potential conflicts.",
"error_code": "#pragma HLS pipeline // Incorrectly placed inside the if statement // Faulty Code\n#pragma HLS PIPELINE // Original Code",
"error_reason": "Placing the pipeline pragma inside conditional blocks can lead to synthesis challenges or resource conflicts. Ensuring the pragma is correctly placed in the code's structure guarantees efficient hardware synthesis."
}
]
}
]
},
{
"Type": "Data Handling Errors",
"Details": [
{
"error_type": "SHFT",

"description": "Shifting bits by more than the width of the data type can lead to loss of data or undefined behavior.",
"error_examples": [
{
"example_number": 1,
"error_code_description": "Bit shift range exceeds the data width, causing data loss or undefined behavior.",
"error_code": "bias_buf[0][coo].range(11,0)=stream_temp.range(coo*25+11,coo*24); // Faulty Code\nbias_buf[0][coo].range(11,0)=stream_temp.range(coo*24+11,coo*24); // Original Code",
"error_reason": "Shifting by `coo*25+11` exceeds the data width, causing potential data loss. Correcting the shift to `coo*24+11` ensures proper data extraction."
},
{
"example_number": 2,
"error_code_description": "Incorrect bit shift leading to an incorrect array index calculation.",
"error_code": "a[i+(2 << DIM)][j] = tmp; // Faulty Code\na[i+2*DIM][j] = tmp; // Original Code",
"error_reason": "Using `2 << DIM` leads to an exponentially larger shift than intended. Correcting to `2*DIM` ensures the correct index calculation."
}
]
},
{
"error_type": "USE",
"description": "Incorrect type casting or operations that extend the sign bit into more significant bits of the data, leading to incorrect calculations.",
"error_examples": [
{
"example_number": 1,
"error_code_description": "Incorrect type casting leading to reduced data width and potential overflow.",
"error_code": "FIX_16_4 tmp[DEPTH]; // Faulty Code\nFIX_32_12 tmp[DEPTH]; // Original Code",
"error_reason": "Using `FIX_16_4` instead of `FIX_32_12` can lead to overflow and loss of precision. Reverting to the correct type ensures adequate data width."
},
{
"example_number": 2,
"error_code_description": "Sign extension due to incorrect casting affecting the calculation.",
"error_code": "c_tm1[i] = cont_input * (ft*(int)c_tm1[i]) + it*gt; // Faulty Code\nc_tm1[i] = cont_input * (ft*c_tm1[i]) + it*gt; // Original Code",

"error_reason": "Casting `c_tm1[i]` to `int` may extend the sign bit when multiplied by `ft`. Removing the cast ensures correct calculations."
}
]
},
{
"error_type": "IDAP",
"description": "Non-sequential or random access patterns in functions marked with dataflow or streaming interfaces can prevent efficient data streaming and lead to increased latency or throughput issues.",
"error_examples": [
{
"example_number": 1,
"error_code_description": "Incompatible data access pattern for the specified interface type.",
"error_code": "#pragma HLS INTERFACE ap_fifo port=a // Faulty Code\n#pragma HLS INTERFACE bram port=a // Original Code",
"error_reason": "Using `ap_fifo` for non-sequential access leads to inefficiency. Reverting to `bram` ensures the correct interface is used for the access pattern."
},
{
"example_number": 2,
"error_code_description": "Interface mode specified does not support the required data access pattern.",
"error_code": "#pragma HLS interface mode=ap_none port=out\nout.write(accumulators[fold]); // Faulty Code\nout.write(accumulators[fold]); // Original Code",
"error_reason": "Specifying `mode=ap_none` for a port expected to handle streaming data may cause latency issues. Correcting the interface mode ensures efficient data streaming."
}
]
},
{
"error_type": "RAMB",
"description": "Non-burst or random access through an m_axi interface can significantly degrade performance due to the lack of data locality and increased transaction overhead.",
"error_examples": [
{
"example_number": 1,
"error_code_description": "Incorrect interface specification for the access pattern, leading to performance degradation.",
"error_code": "#pragma HLS ARRAY_PARTITION variable=bottom cyclic

dim=1 factor=16 \n #pragma HLS INTERFACE port=bottom m_axi depth=16 // Faulty Code\n#pragma HLS ARRAY_PARTITION variable=bottom cyclic dim=1 factor=16 // Original Code",
"error_reason": "Specifying `m_axi` interface with a depth that does not match the burst access pattern can degrade performance. Ensuring the interface matches the access pattern and partitioning improves performance."
},
{
"example_number": 2,
"error_code_description": "Non-burst random access specified for an m_axi interface, causing inefficient data transactions.",
"error_code": "#pragma HLS INTERFACE m_axi port=a offset=slave // Faulty Code\n#pragma HLS INTERFACE bram port=a // Original Code",
"error_reason": "Using `offset=slave` indicates non-burst access, which is inefficient for `m_axi`. Removing the `offset` parameter or changing the interface type to `bram` ensures burst-capable access and better performance."
}
]
},
{
"error_type": "SMA",
"description": "Scalar values mapped to array interfaces like bram/ap_memory/m_axi/ap_fifo/axis for #pragma HLS interface ",
"error_examples": [
{
"example_number": 1,
"Original Code": "#pragma HLS INTERFACE bram port=a",
"Faulty Code": "#pragma HLS INTERFACE bram port=tmp"
},
{
"example_number": 2,
"Original Code": "#pragma HLS INTERFACE bram port=a",
"Faulty Code": "#pragma HLS INTERFACE s_axilite port=tmp"
}
]
}
]
},
{
"Type": "Compatibility Errors",
"Details": [
{
"error_type": "Unsupported_Data_Types",
"description": "Using data types or operations not supported by the HLS tool.",

"error_examples": [
{
"Unsupported Float Operations": {
"Description": "Floating-point operations that may not be synthesized efficiently or correctly by the HLS tool.",
"Common Operations": ["sqrt", "pow"]
},
"Complex Data Structures": {
"Original Code": "static data_t ram[RAM_SIZE]",
"Faulty Code": "vector<data_t> ram[RAM_SIZE];"
},
"Vector Data Structures": {
"Original Code": "dout_t acc; ",
"Faulty Code": "std::complex<dout_t> acc; "
}
}
]
},
{
"error_type": "Dynamic_Data_Structures",
"description": "Attempting to use dynamically allocated structures, which are typically unsupported in HLS environments.",
"error_examples": {
"Incorrect Usage": "int* array = new int[10];\n delete[] array;",
"Corrected Usage": "int array[10];"
}
},
{
"error_type": "Unsupported_Struct_Definition",
"description": "Structures or unions that contain elements or constructs not supported by HLS.",
"error_examples": {
"Unsupported Struct": {
"Definition": "struct Complex { float* values; };",
"Issue": "Use of pointers within the struct is not supported in HLS."
},
"Supported Struct": {
"Definition": "struct Simple { float values[10]; };",
"Resolution": "Replace pointers with fixed-size arrays if possible."
}
}
},
{
"error_type": "Top_Function_Not_Found",

"description": "The absence of a properly defined top function, crucial for defining the entry point of the design.",
"error_examples": {
"Incorrect Usage": "void loop_functions_error(din_t A[N], din_t B[N], dout_t X[N], dout_t Y[N], dsel_t xlimit, dsel_t ylimit) { \n",
"Corrected Usage": "void loop_functions(din_t A[N], din_t B[N], dout_t X[N], dout_t Y[N], dsel_t xlimit, dsel_t ylimit) { \n",
"Description": "Defining a top function in HLS-C",
"Code Snippet": "Specify the Top Function in the synthesis interface of the project settings. }"
}
},
{
"error_type": "Main Function Definition Conflict",
"description": "The project contains more than one testbench.cpp file, leading to multiple definitions of the main() function.",
"error_examples": [
{
"Type": "Compilation Error",
"Detail": "The compiler reports an error due to multiple definitions of the 'main' function."
},
{
"Type": "Linking Error",
"Detail": "The linker fails to create an executable due to multiple 'main' function definitions."
}
]
}
]
},
{
"Type": "C/C++ Syntax/Logic Errors",
"Details": [
{
"error_type": "misused_==/=",
"description": "Misusing == and = in C/C++ can lead to significant logical errors: == checks for equality between two values, while = assigns the value on its right to the variable on its left. Misuse can alter program flow and data handling, often without immediate error indications, making debugging challenging.",
"error_examples": [
{
"example_number": 1,
"oracle_code": "if(s==\"\") if(sum==n) return true;\n\n",

"buggy_code": "if(s==\"\") if(sum=n) return true; \n\n",
"explanations": "\nThe bug is caused by using '=' (assignment operator) instead of '==' (equality operator) for comparison.\n"
},
{
"example_number": 2,
"oracle_code": "return i == n && hasDigits;\n",
"buggy_code": "return i = n && hasDigits;\n",
"explanations": "\nThe bug is on the last line of the isNumber function, where \"i = n\" should be \"i == n\".\n"
},
{
"example_number": 3,
"oracle_code": "if(liveNeighbors < 2 || liveNeighbors > 3)\n board[i][j] = 0;\n\n else if(liveNeighbors == 3)\n board[i][j] = 1;\n\n",
"buggy_code": "if(liveNeighbors < 2 || liveNeighbors > 3)\n board[i][j] = 0;\n\n else if(liveNeighbors = 3)\n board[i][j] = 1;\n\n",
"explanations": "\nI replaced '==' with '=' in 'liveNeighbors == 3', causing liveNeighbors to be constant.\n"
}
]
},
{
"error_type": "missing_colons",
"description": "Omitting colons in C/C++ can disrupt syntax and structure, leading to compile errors or incorrect logic flow. This mistake often occurs in class definitions, loop constructs, and conditional statements, where the absence of a colon can significantly alter the intended behavior or prevent the code from compiling.",
"error_examples": [
{
"example_number": 1,
"oracle_code": "class Solution {\npublic:\n",
"buggy_code": "\nclass Solution {\npublic\n",
"explanations": "\nMissing a colon after 'public' - it should be 'public:' not 'public'\n"
},
{
"example_number": 2,
"oracle_code": "for(int i=0;i<n;i++){\n temp[(i+k)%n]=nums[i];\n }\n nums=temp;\n }\n};",
"buggy_code": "for(int i=0;i<n;i++){\n temp[(i+k)%n]=nums[i]\n }\n nums=temp;\n }\n};",
"explanations": "\nThe semicolon after for loop causes the loop to do nothing, causing incorrect rotation.\n"

```
            },
            {
              "example_number": 3,
              "oracle_code": "class Solution {\npublic:\n      vector<int> dir = { 0, 1, 0, -1, 0 };
\n",
              "buggy_code": "\nclass Solution {\npublic:\n      vector<int> dir = { 0, 1, 0, -1, 0 }
\n",
              "explanations": "\nI removed the semicolon at the end of the vector initialization.
This leads to a compile error.\n"
            }
          ]
        },
        {
          "error_type": "unclosed_parentheses",
          "description": "Unclosed parentheses in C/C++ can introduce syntax errors or disrupt
logical structures. This usually occurs in conditional expressions, function calls, or loops, where
failing to close a parenthesis leads to compilation errors or unexpected behavior, as the compiler
cannot correctly parse the intended code structure.",
          "error_examples": [
            {
              "example_number": 1,
              "oracle_code": "if(str1+str2==str2+str1)\n                {\n                return
str1.substr(0,gcd(str1.length(),str2.length()));\n            }\n            else{\n                return
\"\";\n            }\n            \n      }\n};",
              "buggy_code": "if((str1+str2==str2+str1)\n                {\n                return
str1.substr(0,gcd(str1.length(),str2.length()));\n            }\n            else{\n                return
\"\";\n            }\n            \n      }\n};\n",
              "explanations": "\nThere's a missing closing parenthesis in the 'if' condition,
causing a syntax error.\n"
            },
            {
              "example_number": 2,
              "oracle_code": "for(int i=0;i<nums.size();i++){\n
prefVector[i]=prefPro;\n            prefPro*=nums[i];\n            }\n",
              "buggy_code": "for(int i=0;i<nums.size();i++{\n
prefVector[i]=prefPro;\n            prefPro*=nums[i];\n            }\n",
              "explanations": "\nThere's a missing closing parenthesis in the 'for' condition.\n"
            },
            {
              "example_number": 3,
              "oracle_code": "ans += (nums[i]*1LL*pow(10,y)+nums[j]);\n",
              "buggy_code": "ans += (nums[i]*1LL*pow(10,y)+nums[j];\n",
              "explanations": "\nThere is a missing closing parenthesis after nums[j] in the line
where ans is calculated.\n"
```

}
]
},
{
"error_type": "illegal_separation",
"description": "Illegal separation in C/C++ refers to incorrect or missing syntactic elements that disrupt code structure, such as semicolons or braces, leading to compile-time errors or logic issues. This error type affects loops, conditionals, and function definitions, where the intended code block or statement separation is compromised, causing unintended behavior or compilation failures.",
"error_examples": [
{
"example_number": 1,
"oracle_code": "void dfs(int i,int ansc,vector<vector<int>>&ans,vector<int>&vis,vector<vector<int>>&res){\n \n vis[i]=1;\n \n for(auto t:ans[i]){\n if(!vis[t]){\n dfs(t,ansc,ans,vis,res);\n }\n }\n \n if(i!=ansc)\n res[i].push_back(ansc);\n }\n",
"buggy_code": "void dfs(int i,int ansc,vector<vector<int>>&ans,vector<int>&vis,vector<vector<int>>&res){\n \n vis[i]=1;\n for(auto t:ans[i]){\n if(!vis[t])\n dfs(t,ansc,ans,vis,res)\n }\n if(i!=ansc)\n res[i].push_back(ansc);\n }\n",
"explanations": "\nI omitted the semicolon at the end of dfs function call resulting in statement separation error.\n"
},
{
"example_number": 2,
"oracle_code": "class Solution {\npublic:\n bool isAnagram(string s, string t) {\n if(s.length()!=t.length())\n return false;\n vector<int> freq(26, 0);\n for(int i =0; i<s.length(); i++){\n freq[s[i]-'a']++;\n freq[t[i]-'a']--;\n }\n for(int i =0; i<freq.size(); i++){\n if(freq[i]!=0)\n return false;\n }\n return true;\n }\n};",
"buggy_code": "\nclass Solution {\npublic:\n bool isAnagram(string s, string t) {\n if(s.length()!=t.length())\n return false;\n vector<int> freq(26, 0);\n for(int i =0; i<s.length(); i++)\n freq[s[i]-'a']++;\n freq[t[i]-'a']--;\n for(int i =0; i<freq.size(); i++)\n if(freq[i]!=0)\n return false;\n\n return true;\n }\n};\n",
"explanations": "\nThe for loop's body is not properly bracketed, causing incorrect element decrements in the vector.\n"
},
{
"example_number": 3,
"oracle_code": "void rotate(vector<int>& nums, int k) {\n int

n=nums.size();\n vector<int> temp(nums.size());\n for(int i=0;i<n;i++){\n temp[(i+k)%n]=nums[i];\n }\n nums=temp;\n }\n};",
"buggy_code": "void rotate(vector<int>& nums, int k) \n int n=nums.size();\n vector<int> temp(nums.size());\n for(int i=0;i<n;i++){\n temp[(i+k)%n]=nums[i];\n }\n nums=temp;\n};\n",
"explanations": "\nThe opening brace for rotate function is missing, causing compilation error due to incorrect structuring.\n"
}
]
},
{
"error_type": "unclosed_string",
"description": "Unclosed strings in programming refer to string literals that are initiated but not properly terminated with a matching quotation mark. This error prevents the code from compiling, as the end of the string cannot be determined, leading to syntax errors. In many programming languages, including C++, strings must be enclosed within either single (') or double (\") quotes, and failing to close a string can cause the compiler to interpret the subsequent code as part of the string, resulting in unexpected behavior or compilation failures.",
"error_examples": [
{
"example_number": 1,
"oracle_code": "for(int i =0; i<freq.size(); i++){\n if(freq[i]!=0)\n return false;\n }\n return true;\n }\n};",
"buggy_code": "for(int i =0; i<freq.size(); i++){\n if(freq[i]!=0)\n return 'false;\n }\n return true;\n }\n};\n",
"explanations": "\nThe string 'false is unclosed which causes a compilation error.\n"
},
{
"example_number": 2,
"oracle_code": "while(i<n and s[i]!=' ')s[i]=tolower(s[i]),i++,l++;\n if(i>0)\n i++;\n if(l>2){\n s[j]=toupper(s[j]);\n }\n \n }\n return s;\n }\n};",
"buggy_code": "while(i<n and s[i]!=' ')s[i]=tolower(s[i]),i++,l++;\n if(i>0)\n string error = \"This string is not closed;\n i++;\n if(l>2){\n s[j]=toupper(s[j]);\n }\n \n }\n return s;\n }\n};\n",
"explanations": "\nThe string defined as \"error\" isn't properly closed which will cause a compilation error.\n"
},
{
"example_number": 3,
"oracle_code": "LOOP_I:for(i=0; i < 20; i++){\n acc = 0;\n

LOOP_J: for(j=0; i < 20; j++){\n acc += A[j] * j;\n }",
"buggy_code": "'LOOP_I:for(i=0; i < 20; i++){\n acc = 0;\n LOOP_J: for(j=0; i < 20; j++){\n acc += A[j] * j;\n }",
"explanations": "\nThe string 'LOOP_I' is incorrectly defined as "LOOP_I', which causes a syntax error.\n"
}
]
},
{
"error_type": "illegal_comment",
"description": "Illegal comments in programming refer to the misuse or syntax errors in comment notation that can lead to compilation errors or unintended code behavior. Comments are meant for annotating code and are ignored by compilers, but incorrect usage can affect the execution or interpretation of code. Common issues include unclosed comment blocks, using incorrect symbols for commenting, or placing comment tokens in a way that disrupts code syntax. This can lead to parts of code being mistakenly commented out or compilation failures.",
"error_examples": [
{
"example_number": 1,
"oracle_code": "class Solution {\npublic:\n int mod=1e9+7;\n vector<int>prime;\n int mp[45];\n long long dp[31][1<<12];\n int find(int n)\n {\n for(int i=2;i<=sqrt(n);i++)\n {\n if(n%i==0)\n {\n return false;\n }\n }\n return true;\n }\n long long find(int value,int mask)\n {\n if(value>30)\n {\n return (mask>0);\n }\n if(dp[value][mask]!=-1)\n {\n return dp[value][mask]%mod;\n }\n int new_mask=0;\n long long product=1;\n for(int i=0;i<prime.size();i++)\n {\n if((value%prime[i])==0)\n {\n product*=prime[i];\n new_mask|=(1<<i);\n }\n }\n long long ans=find(value+1,mask);\n if(product==value&&mp[value]&&((mask&new_mask)==0))\n {\n ans+=(find(value+1,mask|new_mask)*mp[value]);\n }\n return dp[value][mask]=ans%mod;\n }\n int numberOfGoodSubsets(vector<int>& nums) \n {\n memset(dp,-1,sizeof(dp));\n for(int i=2;i<=30;i++)\n {\n if(find(i))\n {\n prime.push_back(i);\n }\n }\n for(auto it:nums)\n {\n mp[it]++;\n }\n long long val=find(2,0)%mod;\n long one=1;\n for(int i=0;i<mp[1];i++)\n {\n one=(one*2)%mod;\n }\n val=(val*one)%mod;\n return val;\n }\n};",
"buggy_code": "\nclass Solution {\npublic:\n int mod=1e9+7;\n vector<int>prime;\n \n long long dp[31][1<<12];\n int find(int n)\n {\n for(int i=2;i<=sqrt(n);i++)\n {\n if(n%i==0)\n {\n return false;\n }\n }\n return true;\n }\n long long find(int value,int mask)\n {\n if(value>30)\n {\n return

(mask>0);\n }\n if(dp[value][mask]!=-1)\n {\n return dp[value][mask]%mod;\n }\n int new_mask=0;\n long long product=1;\n for(int i=0;i<prime.size();i++)\n {\n if((value%prime[i])==0)\n {\n product*=prime[i];\n new_mask|=(1<<i);\n }\n }\n long long ans=find(value+1,mask);\n if(product==value&&(mask&new_mask)==0)\n {\n ans+=(find(value+1,mask|new_mask));\n }\n return dp[value][mask]=ans%mod;\n }\n int numberOfGoodSubsets(vector<int>& nums) \n {\n memset(dp,-1,sizeof(dp));\n for(int i=2;i<=30;i++)\n {\n if(find(i))\n {\n prime.push_back(i);\n }\n }\n for(auto it:nums)\n {\n \n }\n long long val=find(2,0)%mod;\n long one=1;\n for(int i=0;i<mp[1];i++)\n {\n one=(one*2)%mod;\n }\n val=(val*one)%mod;\n return val;\n }\n};\n",
"explanations": "\nThe count array 'mp' has been accidentally commented, causing errors in the implementation.\n"
},
{
"example_number": 2,
"oracle_code": "class Solution {\npublic:\n int sumDigits(int num) {\n int sum = 0;\n while(num) {\n sum += num%10;\n num /= 10;\n }\n return sum;\n }\n \n int getLucky(string s, int k) {\n int sum = 0 ; \n for(auto it: s){\n int value = it - 'a' + 1;\n if(value>=10){\n sum+= value%10 + value/10; //since digit is two place only and we need all the digits(_ _)\n }else\n sum = sum + value; \n }\n\n k--;//since I did the sum once;\n while(k-- && sum>=10){//further transformations starts here\n sum = sumDigits(sum);\n } \n return sum;\n }\n};",
"buggy_code": "\nclass Solution {\npublic:\n int sumDigits(int num) {\n int sum = 0;\n while(num) {\n sum += num%10;\n num /= 10;\n }\n return sum;\n }\n \n int getLucky(string s, int k) {\n int sum = 0 ; \n for(auto it: s){\n int value = it - 'a' + 1;\n \n sum+= value%10 + value/10; //since digit is two place only and we need all the digits(_ _)\n \n sum = sum + value; \n }\n\n k--;//since I did the sum once;\n while(k-- && sum>=10){//further transformations starts here\n sum = sumDigits(sum);\n } \n return sum;\n }\n};\n",
"explanations": "\nThe commented if-else logic in the getLucky function inappropriately increases the sum for single digit values.\n"
},
{
"example_number": 3,
"oracle_code": "// cout<<n<<\" \";\n",
"buggy_code": "/ cout<<n<<\" \";\n",
"explanations": "\nUsed single slash '/' instead of double '//' for a comment,

causing a syntax error.\n"
}
]
},
{
"error_type": "faulty_indexing",
"description": "Faulty indexing in programming refers to accessing elements of an array, list, or similar data structures using indices that are either out of bounds or incorrect based on the context of the operation. This can lead to runtime errors such as \"index out of bounds\" exceptions or accessing incorrect data, which can disrupt the application's logic and flow.",
"error_examples": [
{
"example_number": 1,
"oracle_code": "class Solution {\npublic:\n vector<int> productExceptSelf(vector<int>& nums) {\n vector<int>prefVector(nums.size(),1);\n vector<int>suffVector(nums.size(),1);\n int prefPro=1;\n for(int i=0;i<nums.size();i++){\n prefVector[i]=prefPro;\n prefPro*=nums[i];\n }\n int suffPro=1;\n for(int i=nums.size()-1;i>=0;i--){\n suffVector[i]=suffPro;\n suffPro*=nums[i];\n }\n vector<int>ans(nums.size());\n for(int i=0;i<nums.size();i++){\n ans[i]=prefVector[i]*suffVector[i];\n }\n return ans;\n }\n};",
"buggy_code": "\nclass Solution {\npublic:\n vector<int> productExceptSelf(vector<int>& nums) {\n vector<int>prefVector(nums.size(),1);\n vector<int>suffVector(nums.size(),1);\n int prefPro=1;\n for(int i=0;i<=nums.size();i++){\n prefVector[i]=prefPro;\n prefPro*=nums[i];\n }\n int suffPro=1;\n for(int i=nums.size();i>=0;i--){\n suffVector[i]=suffPro;\n suffPro*=nums[i];\n }\n vector<int>ans(nums.size());\n for(int i=0;i<nums.size();i++){\n ans[i]=prefVector[i]*suffVector[i];\n }\n return ans;\n }\n};\n",
"explanations": "\n'nums' array is accessed beyond its size in the prefix and suffix product calculation.\n"
},
{
"example_number": 3,
"oracle_code": "class Solution {\npublic:\n int maxWidthOfVerticalArea(vector<vector<int>>& points) {\n set<int> x;\n int res = 0;\n for (auto &p : points)\n x.insert(p[0]);\n for (auto it = next(begin(x)); it != end(x); ++it)\n res = max(res, *it - *prev(it));\n return res;\n}\n};",
"buggy_code": "\nclass Solution {\npublic:\n int maxWidthOfVerticalArea(vector<vector<int>>& points) {\n set<int> x;\n int res = 0;\n for (auto &p : points)\n x.insert(p[2]);\n for (auto it = next(begin(x)); it != end(x); ++it)\n res = max(res, *it - *prev(it));\n return res;\n}\n};\n",

"explanations": "\nThe vector 'p' doesn't have a third element, causing an out-of-bounds indexing error.\n"
},
{
"example_number": 4,
"oracle_code": "class Solution {\npublic:\n vector<int> findDisappearedNumbers(vector<int>& nums) {\n int n = nums.size();\n int i = 0;\n while (i<n) {\n if (nums[i] != nums[nums[i]-1] && i != nums[i]-1)\n swap(nums[i], nums[nums[i]-1]);\n else i++;\n }\n \n vector<int> res;\n for (int i=0; i<n; i++) {\n if (nums[i] != i+1) res.push_back(i+1);\n }\n\n return res;\n }\n};",
"buggy_code": "\nclass Solution {\npublic:\n vector<int> findDisappearedNumbers(vector<int>& nums) {\n int n = nums.size();\n int i = 0;\n while (i<n) {\n if (nums[i] != nums[nums[i]-1] && i != nums[i]-1)\n swap(nums[i], nums[nums[i]]);\n else i++;\n }\n \n vector<int> res;\n for (int i=0; i<n; i++) {\n if (nums[i] != i+1) res.push_back(i+1);\n }\n\n return res;\n }\n};\n",
"explanations": "\nThe bug is in the swap statement. nums[nums[i]] will have an index value of nums[i] which might exceed the size of the array.\n"
},
{
"example_number": 5,
"oracle_code": "class Solution {\npublic:\n int longestSubarray(vector<int>& nums) {\n int n = nums.size();\n int left = 0, right, zeros = 0, ans = 0;\n\n for (right = 0; right < n; right++) {\n if (nums[right] == 0){\n zeros++;\n }\n if (zeros > 1 && nums[left++] == 0){\n zeros--;\n }\n ans = max(ans, right - left);\n }\n return ans; \n }\n};",
"buggy_code": "\nclass Solution {\npublic:\n int longestSubarray(vector<int>& nums) {\n int n = nums.size();\n int left = 0, right, zeros = 0, ans = 0;\n\n for (right = 0; right <= n; right++) {\n if (nums[right] == 0){\n zeros++;\n }\n if (zeros > 1 && nums[left++] == 0){\n zeros--;\n }\n ans = max(ans, right - left);\n }\n return ans; \n }\n};\n",
"explanations": "\nThe loop iterates beyond the size of the vector, causing array index out of bound errors.\n"
}
]
},
{
"error_type": "undefined_objects",
"description": "Undefined objects in programming occur when a variable or object is used in the code without being properly declared or initialized, leading to runtime errors or undefined behavior.",

"error_examples": [
{
"example_number": 1,
"oracle_code": "class Solution {\npublic:\n int nthUglyNumber(int n) \n {\n vector<int>dp(n);\n dp[0]=1;\n int p1=0;\n int p2=0;\n int p3=0;\n for(int i=1;i<n;i++)\n {\n int two=dp[p1]*2;\n int three=dp[p2]*3;\n int five=dp[p3]*5;\n dp[i]=min(two,min(three,five));\n if(dp[i]==two)\n p1++;\n if(dp[i]==three)\n p2++;\n if(dp[i]==five)\n p3++;\n }\n return dp[n-1];\n \n }\n};",
"buggy_code": "\nclass Solution {\npublic:\n int nthUglyNumber(int n) \n {\n vector<int>dp(n);\n dp[0]=1;\n int p1=0;\n int p2=0;\n int p3=0;\n for(int i=1;i<=n;i++)\n {\n int two=dp[p1]*2;\n int three=dp[p2]*3;\n int five=dp[p3]*5;\n dp[i]=min(two,min(three,five));\n if(dp[i]==two)\n p1++;\n if(dp[i]==three)\n p2++;\n if(dp[i]==five)\n p3++;\n }\n return dp[n];\n }\n};\n",
"explanations": "\nVector dp has n elements, but loop runs n+1 times, causing an out-of-bound error.\n"
},
{
"example_number": 2,
"oracle_code": "class Solution {\npublic:\n\n // declare a dp\n\n int dp[105][10005];\n\n int helper(vector<int>& arr, int i, int n, int sum)\n {\n // base case\n\n if(sum == 0)\n {\n return 0;\n }\n\n if(i >= n || sum < 0)\n {\n return INT_MAX - 1000;\n }\n\n // if already calculated\n\n if(dp[i][sum] != -1)\n {\n return dp[i][sum];\n }\n\n // at each ith element we have two options either include or exclude\n\n int mini = INT_MAX;\n\n // inclusion part\n\n if(arr[i] <= sum)\n {\n mini = min(mini, 1 + helper(arr, i, n, sum - arr[i]));\n }\n\n // exclusion part\n\n mini = min(mini, helper(arr, i + 1, n, sum));\n\n // store the res in dp\n\n return dp[i][sum] = mini;\n }\n\n int numSquares(int n) {\n\n vector<int> arr;\n\n // find all the squares between 1 to n\n\n for(int i = 1; i * i <= n; i++)\n {\n arr.push_back(i * i);\n }\n\n // initialize dp with -1\n\n memset(dp, -1, sizeof(dp));\n\n return helper(arr, 0, arr.size(), n);\n }\n};",
"buggy_code": "\nclass Solution {\npublic:\n int dp[105][10005];\n \n int helper(int i, int n, int sum)\n {\n vector<int>& arr;\n\n if(sum == 0)\n {\n return 0;\n }\n\n if(i >= n || sum < 0)\n {\n return INT_MAX - 1000;\n }\n\n if(dp[i][sum] != -1)\n {\n return dp[i][sum];\n }\n\n int mini = INT_MAX;\n\n if(arr[i] <= sum)\n {\n mini = min(mini, 1 + helper(i, n, sum - arr[i]));\n }\n\n mini = min(mini, helper(i + 1, n, sum));\n\n return dp[i][sum] = mini;\n }\n\n int numSquares(int n) {\n\n vector<int> arr;\n\n

for(int i = 1; i * i <= n; i++)\n {\n arr.push_back(i * i);\n }\n\n memset(dp, -1, sizeof(dp));\n\n return helper(0, arr.size(), n);\n }\n};\n",
"explanations": "\nThe vector 'arr' is declared in 'helper' but not defined, causing an undefined behavior.\n"
},
{
"example_number": 3,
"oracle_code": "class Solution {\npublic:\n bool buddyStrings(string s, string p) {\n int n=s.length(), m=p.length();\n if(n!=m) return false;\n \n vector<int> freq(26,0);\n \n int first=-1, second=-1;\n for(int i=0;i<n;i++){\n if(s[i] != p[i]){\n if(first==-1){\n first=i;\n }else if(second==-1){\n second=i;\n }else{\n return false; // more than 2 mismatching characters\n }\n }\n freq[s[i]-'a']++;\n }\n \n\t\t// similar strings\n if(first == -1 && second == -1){\n for(int i=0;i<26;i++) if(freq[i] > 1) return true; // can swap similar characters\n return false; // all characters occur only once\n }else if(second == -1){\n return false; // exactly 1 mismatch\n }\n \n return (s[first]==p[second] && s[second]==p[first]); // check similarity on 2 mismatched indexes\n }\n};",
"buggy_code": "\nclass Solution {\npublic:\n bool buddyStrings(string s, string p) {\n int n=s.length(), m=p.length();\n if(n!=m) return false;\n \n vector<int> freq(26,0);\n \n int first=-1, second=-1;\n char *undefinedObject;\n\n for(int i=0;i<n;i++){\n if(s[i] != p[i]){\n if(first==-1){\n first=i;\n }else if(second==-1){\n second=i;\n }else{\n return false; // more than 2 mismatching characters\n }\n }\n freq[undefinedObject[i]-'a']++;\n }\n \n \n if(first == -1 && second == -1){\n for(int i=0;i<26;i++) if(freq[i] > 1) return true; // can swap similar characters\n return false; // all characters occur only once\n }else if(second == -1){\n return false; // exactly 1 mismatch\n }\n \n return (s[first]==p[second] && s[second]==p[first]); // check similarity on 2 mismatched indexes\n }\n};\n",
"explanations": "\nAdding values to non-existent object \"undefinedObject\" causes the program to go haywire.\n"
}
]
},
{
"error_type": "undefined_methods",
"description": "Undefined methods in programming refer to attempts to invoke functions or methods that have not been declared or defined within the scope of use, leading to compile-time errors.",
"error_examples": [

{
"example_number": 1,
"oracle_code": "class Solution {\n int res,c = 0,n,vis[201]={0};\n void dfs(vector<vector<int>>& isConnected,int start)\n {\n vis[start] = true;c++;\n for(int connection = 0;connection<n;connection++)\n {\n if(!vis[connection] and isConnected[start][connection]) dfs(isConnected,connection);\n }\n }\npublic:\n int findCircleNum(vector<vector<int>>& isConnected)\n {\n n = isConnected.size();\n for(int i = 0;i<n;i++) if(!vis[i]) c=0,dfs(isConnected,i),res++;;\n return res; \n }\n};",
"buggy_code": "\nclass Solution {\n int res,c = 0,n,vis[201]={0};\n void dfs(vector<vector<int>>& isConnected,int start)\n {\n vis[start] = true;c++;\n for(int connection = 0;connection<n;connection++)\n {\n if(!vis[connection] and isConnected[start][connection]) dfs(isConnected,connection);\n }\n }\npublic:\n int findCircleNum(vector<vector<int>>& isConnected)\n {\n n = isConnected.size();\n for(int i = 0;i<n;i++) if(!vis[i]) c=0,dfs(isConnected,i),incrementRes();\n return res; \n }\n\n void incrementRes();\n};\n",
"explanations": "\nThe \"incrementRes()\" function is declared but not defined anywhere in the class.\n"
},
{
"example_number": 2,
"oracle_code": "class Solution {\npublic:\n\n // declare a dp\n\n int dp[105][10005];\n\n int helper(vector<int>& arr, int i, int n, int sum)\n {\n // base case\n\n if(sum == 0)\n {\n return 0;\n }\n\n if(i >= n || sum < 0)\n {\n return INT_MAX - 1000;\n }\n\n // if already calculated\n\n if(dp[i][sum] != -1)\n {\n return dp[i][sum];\n }\n\n // at each ith element we have two options either include or exclude\n\n int mini = INT_MAX;\n\n // inclusion part\n\n if(arr[i] <= sum)\n {\n mini = min(mini, 1 + helper(arr, i, n, sum - arr[i]));\n }\n\n // exclusion part\n\n mini = min(mini, helper(arr, i + 1, n, sum));\n\n // store the res in dp\n\n return dp[i][sum] = mini;\n }\n\n int numSquares(int n) {\n\n vector<int> arr;\n\n // find all the squares between 1 to n\n\n for(int i = 1; i * i <= n; i++)\n {\n arr.push_back(i * i);\n }\n\n // initialize dp with -1\n\n memset(dp, -1, sizeof(dp));\n\n return helper(arr, 0, arr.size(), n);\n }\n};",
"buggy_code": "\nclass Solution {\npublic:\n\n // declare a dp\n\n int dp[105][10005];\n\n int helper(vector<int>& arr, int i, int n, int sum)\n {\n // base case\n\n if(sum == 0)\n {\n return 0;\n }\n\n if(i >= n || sum < 0)\n {\n return INT_MAX - 1000;\n }\n\n // if already calculated\n\n if(dp[i][sum] != -1)\n {\n return dp[i][sum];\n }\n\n // at each ith element we have two options either include or

exclude\n\n int mini = INT_MAX;\n\n // inclusion part\n\n if(arr[i] <= sum)\n {\n mini = min(mini, 1 + helper(arr, i, n, sum - arr[i]));\n }\n\n // exclusion part\n\n mini = min(mini, helper(arr, i + 1, n, sum));\n\n // store the res in dp\n\n return dp[i][sum] = mini;\n }\n\n int numSquares(int n) {\n\n vector<int> arr;\n\n // find all the squares between 1 to n\n\n for(int i = 1; i * i <= n; i++)\n {\n arr.push_back(i * i);\n }\n\n // initialize dp with -1\n\n this->initializeDP();\n\n return helper(arr, 0, arr.size(), n);\n }\n\n};\n",
"explanations": "\nThe method 'initializeDP' is declared to be used but it's not defined anywhere in the code.\n"
},
{
"example_number": 3,
"oracle_code": "vector<int> id;\n vector<int>rating;\n for(int i=0; i<rating.size(); i++){\n for(int j=i+1; j<rating.size(); j++){\n if(rating[i] < rating[j]){\n swap(rating[i], rating[j]);\n swap(id[i], id[j]);\n }\n }\n }\n\n for(int i=0; i<rating.size(); i++){\n for(int j=i+1; j<rating.size(); j++){\n if(rating[i] == rating[j] && id[i] < id[j]){\n swap(id[i], id[j]);\n }\n }\n }\n\n return id;\n }\n};",
"buggy_code": "vector<int> id;\n vector<int>rating;\n for(int i=0; i<rating.size(); i++){\n for(int j=i+1; j<rating.size(); j++){\n if(rating[i] < rating[j]){\n swap(rating[i], rating[j]);\n swap(id[i], id[j]);\n }\n }\n }\n\n for(int i=0; i<rating.size(); i++){\n for(int j=i+1; j<rating.size(); j++){\n if(rating[i] == rating[j] && id[i] < id[j]){\n swap(id[i], id[j]);\n }\n }\n }\n\n vector<int> result = sortByRating(id, rating);\n\n return result;\n }\n\n};\n",
"explanations": "\nsortByRating method is called but it's not defined within the class.\n"
}
]
},
{
"error_type": "illegal_keywords",
"description": "Illegal keywords in programming refer to the use of syntax or tokens that are not recognized by the programming language being used, leading to syntax errors.",
"error_examples": [
{
"example_number": 1,
"oracle_code": "class FrequencyTracker {\npublic:\n unordered_map<int,int> x,y;\n void add(int number) {\n x[number]++;\n if(x[number]!=1) y[x[number]-1]--;\n y[x[number]]++;\n }\n void deleteOne(int number) {\n if(x[number]!=0){\n x[number]--;\n

y[x[number]+1]--;\n y[x[number]]++;\n }\n }\n bool hasFrequency(int frequency) {\n if(y[frequency]!=0) return true;\n return false;\n }\n};",
"buggy_code": "\nclass FrequencyTracker {\npublic:\n unordered_map<int,int> x,y;\n def add(number): \n x[number]++;\n if(x[number]!=1) y[x[number]-1]--;\n y[x[number]]++;\n def deleteOne(number): \n if(x[number]!=0){\n x[number]--;\n y[x[number]+1]--;\n y[x[number]]++;\n }\n def hasFrequency(frequency): \n if(y[frequency]!=0) return true;\n return false;\n};\n",
"explanations": "\nThe bug is the use of 'def' keyword from Python to define methods, unrecognizable to C++.\n"
},
{
"example_number": 2,
"oracle_code": "class Solution {\npublic:\n int sumOfMultiples(int n,int count=0) {\n \n \n for (int i=1;i<=n;i++)\n {\n if (i%3==0 || i%5==0 ||i%7==0)\n count+=i;\n }\n return count;\n\n }\n};",
"buggy_code": "\nclass Solution {\npublic:\n let sumOfMultiples = (n, count=0) =>{\n \n \n for (let i=1;i<=n;i++)\n {\n if (i%3==0 || i%5==0 ||i%7==0)\n count+=i;\n }\n return count;\n\n }\n};\n",
"explanations": "\nThe bug is using javascript keywords like \"let\" and arrow functions in a C++ program.\n"
},
{
"example_number": 3,
"oracle_code": "class MyStack {\npublic:\n queue<int> q;\n MyStack() {\n\n }\n \n void push(int x) {\n q.push(x);\n for(int i=0; i<q.size()-1; i++){\n q.push(q.front());\n q.pop();\n }\n }\n \n int pop() {\n int a= q.front();\n q.pop();\n return a;\n }\n \n int top() {\n return q.front();\n }\n \n bool empty() {\n return q.empty();\n }\n};",
"buggy_code": "\nclass MyStack {\npublic:\n queue<int> q;\n MyStack() {\n\n }\n \n void add(int x) { \n q.push(x);\n for(int i=0; i<q.size()-1; i++){\n q.push(q.front());\n q.pop();\n }\n }\n \n int pop() {\n int a= q.front();\n q.pop();\n return a;\n }\n \n int top() {\n return q.front();\n }\n \n bool empty() {\n return q.empty();\n }\n};\n",
"explanations": "\nThe bug is changing method name from push to add which is not a defined method in queue.\n"
}
]
},

{
"error_type": "condition_error",
"description": "Condition errors in programming occur when the logical checks or conditions within the code are incorrect or misaligned with the intended logic, leading to unexpected behavior or incorrect computations. These errors can be syntactically correct but semantically incorrect, making them harder to debug as they do not typically produce compilation errors.",
"error_examples": [
{
"example_number": 1,
"oracle_code": "class Solution {\npublic:\n int nthUglyNumber(int n) \n {\n vector<int>dp(n);\n dp[0]=1;\n int p1=0;\n int p2=0;\n int p3=0;\n for(int i=1;i<n;i++)\n {\n int two=dp[p1]*2;\n int three=dp[p2]*3;\n int five=dp[p3]*5;\n dp[i]=min(two,min(three,five));\n if(dp[i]==two)\n p1++;\n if(dp[i]==three)\n p2++;\n if(dp[i]==five)\n p3++;\n }\n return dp[n-1];\n \n }\n};",
"buggy_code": "\nclass Solution {\npublic:\n int nthUglyNumber(int n) \n {\n vector<int>dp(n);\n dp[0]=1;\n int p1=0;\n int p2=0;\n int p3=0;\n for(int i=1;i<=n;i++)\n {\n int two=dp[p1]*2;\n int three=dp[p2]*3;\n int five=dp[p3]*5;\n dp[i]=min(two,min(three,five));\n if(dp[i]==two)\n p1++;\n if(dp[i]==three)\n p2++;\n if(dp[i]==five)\n p3++;\n }\n return dp[n-1];\n \n }\n};\n",
"explanations": "\nThe loop counter i should run from 1 to n-1, not 1 to n.\n"
},
{
"example_number": 2,
"oracle_code": "class Solution {\npublic:\n int mod=1e9+7;\n long helper(int n, vector<long>& dp){\n if(n==1) return 1;\n if(n==2) return 2;\n if(n==3) return 5;\n\n if(dp[n]!=-1) return dp[n]%mod;\n\n long smallAns1=0, smallAns2=0;\n smallAns1=helper(n-1,dp)%mod;\n smallAns2=helper(n-3,dp)%mod;\n\n dp[n]=(smallAns1*2+smallAns2)%mod;\n return dp[n]%mod;\n }\n \n int numTilings(int n) {\n vector<long> dp(n+1,-1);\n long ans=helper(n,dp)%mod;\n return (int)ans; \n }\n};",
"buggy_code": "\nclass Solution {\npublic:\n int mod=1e9+7;\n long helper(int n, vector<long>& dp){\n if(n==1) return 1;\n if(n==2) return 2;\n if(n==3) return 5;\n\n if(dp[n]!=-1) return dp[n]%mod;\n\n long smallAns1=0, smallAns2=0;\n smallAns1=helper(n-1,dp)%mod;\n smallAns2=helper(n-4,dp)%mod;\n\n dp[n]=(smallAns1*2+smallAns2)%mod;\n return dp[n]%mod;\n }\n \n int numTilings(int n) {\n vector<long> dp(n+1,-1);\n long ans=helper(n,dp)%mod;\n return (int)ans; \n }\n};\n",

"explanations": "\nChanging the recursive call from helper(n-3, dp) to helper(n-4, dp) disrupts the whole logic.\n"
},
{
"example_number": 3,
"oracle_code": "class Solution {\npublic:\n string addStrings(string a, string b) {\n string ret = \"\" ;\n int carry = 0, i = a.size() - 1, j = b.size() - 1;\n while(i >= 0 || j >= 0){\n int x = i >= 0 ? a[i--] - '0' : 0 ;\n int y = j >= 0 ? b[j--] - '0' : 0 ;\n\n int sm = x + y + carry ;\n ret.push_back('0' + (sm % 10)) ;\n carry = sm/10;\n }\n if(carry)\n ret.push_back('0' + carry) ;\n \n reverse(ret.begin(), ret.end()) ;\n\n return ret ;\n }\n};",
"buggy_code": "\nclass Solution {\npublic:\n string addStrings(string a, string b) {\n string ret = \"\" ;\n int carry = 0, i = a.size() - 1, j = b.size() - 1;\n while(i >= 0 || j >= 0){\n int x = i >= 0 ? a[i--] - '0' : 0 ;\n int y = j >= 0 ? b[j--] - '0' : 0 ;\n\n int sm = x + y + carry ;\n ret.push_back('0' + (sm % 10)) ;\n carry = sm/10;\n }\n ret.push_back('0' + carry) ;\n \n reverse(ret.begin(), ret.end()) ;\n\n return ret ;\n }\n};\n",
"explanations": "\nThe \"if\" statement checking if there's a remaining carry to add to the string is removed. This can lead to incorrect string addition results, especially when the last two numbers have a carry. \n"
}
]
},
{
"error_type": "operation_error",
"description": "Operation errors in programming occur when the execution of operations leads to incorrect results due to mistakes in the use of operators, incorrect algorithm implementations, or misuse of functions and methods. These errors can lead to unintended behavior or incorrect data processing, which can be critical depending on the context of the application.",
"error_examples": [
{
"example_number": 1,
"oracle_code": "class Solution {\npublic:\n vector<int> productExceptSelf(vector<int>& nums) {\n vector<int>prefVector(nums.size(),1);\n vector<int>suffVector(nums.size(),1);\n int prefPro=1;\n for(int i=0;i<nums.size();i++){\n prefVector[i]=prefPro;\n prefPro*=nums[i];\n }\n int suffPro=1;\n for(int i=nums.size()-1;i>=0;i--){\n suffVector[i]=suffPro;\n suffPro*=nums[i];\n }\n vector<int>ans(nums.size());\n for(int i=0;i<nums.size();i++){\n ans[i]=prefVector[i]*suffVector[i];\n }\n return ans;\n }\n};",

"buggy_code": "\nclass Solution {\npublic:\n vector<int> productExceptSelf(vector<int>& nums) {\n vector<int>prefVector(nums.size(),1);\n vector<int>suffVector(nums.size(),1);\n int prefPro=1;\n for(int i=0;i<nums.size();i++){\n prefVector[i]=prefPro;\n prefPro*=nums[i];\n }\n int suffPro=1;\n for(int i=nums.size()-1;i>=0;i--){\n suffVector[i]=suffPro;\n suffPro*=nums[i+1];\n }\n vector<int>ans(nums.size());\n for(int i=0;i<nums.size();i++){\n ans[i]=prefVector[i]*suffVector[i];\n }\n return ans;\n }\n};\n",
"explanations": "\nThe bug is in the second loop, suffPro multiplies nums[i+1], potentially accessing invalid memory.\n"
},
{
"example_number": 2,
"oracle_code": "class Solution {\npublic:\n bool buddyStrings(string s, string p) {\n int n=s.length(), m=p.length();\n if(n!=m) return false;\n \n vector<int> freq(26,0);\n \n int first=-1, second=-1;\n for(int i=0;i<n;i++){\n if(s[i] != p[i]){\n if(first==-1){\n first=i;\n }else if(second==-1){\n second=i;\n }else{\n return false; // more than 2 mismatching characters\n }\n }\n freq[s[i]-'a']++;\n }\n \n\t\t// similar strings\n if(first == -1 && second == -1){\n for(int i=0;i<26;i++) if(freq[i] > 1) return true; // can swap similar characters\n return false; // all characters occur only once\n }else if(second == -1){\n return false; // exactly 1 mismatch\n }\n \n return (s[first]==p[second] && s[second]==p[first]); // check similarity on 2 mismatched indexes\n }\n};",
"buggy_code": "\nclass Solution {\npublic:\n bool buddyStrings(string s, string p) {\n int n=s.length(), m=p.length();\n if(n!=m) return false;\n \n vector<int> freq(26,0);\n \n int first=-1, second=-1;\n for(int i=0;i<n;i++){\n if(s[i] != p[i]){\n if(first==-1){\n first=i;\n }else if(second==-1){\n second=i;\n }else{\n return false; // more than 2 mismatching characters\n }\n }\n freq[s[i]-'z']++;\n }\n \n\t\t// similar strings\n if(first == -1 && second == -1){\n for(int i=0;i<26;i++) if(freq[i] > 1) return true; // can swap similar characters\n return false; // all characters occur only once\n }else if(second == -1){\n return false; // exactly 1 mismatch\n }\n \n return (s[first]==p[second] && s[second]==p[first]); // check similarity on 2 mismatched indexes\n }\n};\n",
"explanations": "\nThe ASCII subtraction is incorrect, should be 'a' not 'z', leading to incorrect counting.\n"
},
{
"example_number": 3,

"oracle_code": "class Solution {\npublic:\n bool isRectangleCover(vector<vector<int>>& rectangles) {\n map<pair<int,int>,int>m;\n for(auto it:rectangles){\n m[{it[0],it[1]}]++;\n m[{it[2],it[3]}]++;\n m[{it[0],it[3]}]--;\n m[{it[2],it[1]}]--;\n }\n int cnt=0;\n for(auto it=m.begin();it!=m.end();it++){\n if(abs(it->second)==1){\n cnt++;\n }\n else if(abs(it->second)!=1&&it->second!=0){\n return false;\n }\n }\n return cnt==4;\n }\n};",
"buggy_code": "\nclass Solution {\npublic:\n bool isRectangleCover(vector<vector<int>>& rectangles) {\n map<pair<int,int>,int>m;\n for(auto it:rectangles){\n m[{it[0],it[1]}]++;\n m[{it[2],it[3]}]++;\n m[{it[0],it[3]}]++;\n m[{it[2],it[1]}]++;\n }\n int cnt=0;\n for(auto it=m.begin();it!=m.end();it++){\n if(abs(it->second)==1){\n cnt++;\n }\n else if(abs(it->second)!=1&&it->second!=0){\n return false;\n }\n }\n return cnt==4;\n }\n};\n",
"explanations": "\nThe coordinates of the corners are incremented twice, instead of once, causing incorrect checking.\n"
}
]
},
{
"error_type": "Pointer",
"description": "Pointer errors are typically prohibited unless they are statically analyzable. This restriction exists because the dynamic nature of pointers, particularly those involving heap memory or complex referencing and dereferencing, cannot be effectively translated into hardware constructs. As a result, developers must manually convert pointer operations to array operations to adhere to HLS tool constraints.",
"error_examples": [
{
"example_number": 1,
"oracle_code": "#include \"ram.h\"\n\n#define RAM_SIZE (1 << ADDR_WIDTH)\n\nvoid ram(data_t data, addr_t addr, bool_t we, data_t& q) {\n\n static data_t ram[RAM_SIZE];\n\n if (we) {\n ram[addr] = data;\n }\n\n q = ram[addr];\n}\n",
"buggy_code": "#include \"ram.h\"\n\n#define RAM_SIZE (1 << ADDR_WIDTH)\n\nvoid ram(data_t data, addr_t addr, bool_t we, data_t& q) {\n\n static data_t *ram[RAM_SIZE];\n\n if (we) {\n *ram[addr] = data;\n }\n\n q = *ram[addr];\n}\n",
"explanations": "The \"buggy_code\" fails because it uses an uninitialized pointer 'static data_t *ram[RAM_SIZE]', which is unsafe and inappropriate for hardware synthesis in HLS, where memory should be statically allocated and pointers are generally avoided."
},
{
"example_number": 2,
"oracle_code": "void adv_fsm(bit_t reset, bit_t x, bit_t* z) {\n enum states

{ IDLE=0, S1=1, S10=2 };\n static states present_state = IDLE;\n states next_state = present_state; \n\n bit_t z_internal = 0;\n\n #pragma HLS INTERFACE ap_none port=reset\n #pragma HLS INTERFACE ap_none port=x\n #pragma HLS INTERFACE ap_none port=z\n #pragma HLS INTERFACE ap_ctrl_none port=return\n\n switch (present_state) {\n case IDLE:\n if (x == 1) {\n next_state = S1;\n } else {\n next_state = IDLE;\n }\n z_internal = 0;\n break;\n\n case S1:\n if (x == 0) {\n next_state = S10;\n } else {\n next_state = IDLE;\n }\n z_internal = 0;\n break;\n\n case S10:\n if (x == 1) {\n next_state = IDLE;\n z_internal = 1;\n } else {\n next_state = IDLE;\n z_internal = 0;\n }\n break;\n\n default:\n next_state = IDLE;\n z_internal = 0;\n break;\n }\n\n if (reset) {\n present_state = IDLE;\n } else {\n present_state = next_state;\n }\n\n *z = z_internal;\n}\n",
"buggy_code": "void adv_fsm(bit_t reset, bit_t x, bit_t* z) {\n enum states { IDLE=0, S1=1, S10=2 };\n static states present_state = IDLE;\n states next_state = present_state; \n\n bit_t *z_internal = 0;\n\n #pragma HLS INTERFACE ap_none port=reset\n #pragma HLS INTERFACE ap_none port=x\n #pragma HLS INTERFACE ap_none port=z\n #pragma HLS INTERFACE ap_ctrl_none port=return\n\n switch (present_state) {\n case IDLE:\n if (x == 1) {\n next_state = S1;\n } else {\n next_state = IDLE;\n }\n *z_internal = 0;\n break;\n\n case S1:\n if (x == 0) {\n next_state = S10;\n } else {\n next_state = IDLE;\n }\n *z_internal = 0;\n break;\n\n case S10:\n if (x == 1) {\n next_state = IDLE;\n *z_internal = 1;\n } else {\n next_state = IDLE;\n *z_internal = 0;\n }\n break;\n\n default:\n next_state = IDLE;\n *z_internal = 0;\n break;\n }\n\n if (reset) {\n present_state = IDLE;\n } else {\n present_state = next_state;\n }\n\n *z = *z_internal;\n}\n",
"explanations": "The buggy code introduces a critical error by declaring z_internal as a pointer (int *z_internal = 0) and then attempting to dereference it to assign values, which is forbidden in HLS."
},
{
"example_number": 3,
"oracle_code": "void loop_imperfect(din_t A[N], dout_t B[N]) {\n\n int i,j;\n dint_t acc;\n\n LOOP_I:for(i=0; i < 20; i++){\n acc = 0;\n LOOP_J: for(j=0; j < 20; j++){\n acc += A[j] * j;\n }\n if (i%2 == 0)\n B[i] = acc / 20;\n else\n B[i] = 0;\n }\n\n}\n",
"buggy_code": "void loop_imperfect(din_t A[N], dout_t B[N]) {\n\n int i,j;\n dint_t *acc;\n\n LOOP_I:for(i=0; i < 20; i++){\n *acc = 0;\n LOOP_J: for(j=0; j < 20; j++){\n *acc += A[j] * j;\n }\n if (i%2

== 0)\n B[i] = *acc / 20;\n else\n B[i] = 0;\n }\n\n}\n",
"explanations": "ERROR: System recieved a signal named SIGSEGV and the program has to stop immediately!\nThis signal was generated when a program tries to read or write outside the memory that is allocated for it, or to write memory that can only be read.\nPossible cause of this problem may be: 1) the depth setting of pointer type argument is much larger than it needed; 2)insufficient depth of array argument; 3)null pointer etc."
}
]
}
]
}
]
}